ECE 3TC32: Compilation

A compiler is a program that translates a program from one language into another while preserving its semantics. This module builds one, and the academic content is the pipeline it is made of: *lexeur*, *parseur*, assembler and logic circuits. It starts from the bottom — x86-64 assembly, the memory model, the stack and the calling convention, and the C that compiles down to them — and then climbs back up through formal grammars, ambiguity and parsing to regular languages, automata and lexical analysis.

These notes cover the séances taught so far: séance 1 (introduction générale, premiers pas en x86), séance 2 (rappels de C, suite x86 et libC) and séance 3 (analyse lexicale, with the grammar and parsing material that precedes it). The remaining séances of the séquençage — parseur, portes logiques combinatoires and synchrones, génération de code, and the projet compilateur — are not held yet and get chapters when they are taught.

Titles and technical terms are in French, the language of the course and its exam: each term is glossed in English once, where it is defined, definitions keep the course's own French, and the commentary is in English.

Contents

0 Prérequis **3**
- 0.1 Bases de l'informatique . . . 3
- 0.2 Complément à Deux . . . 3
- 0.3 Code ASCII . . . 4
- 0.4 Unités de Mémoire . . . 4
- 0.5 Rappels de C . . . 5

1 Compilateurs **5**
- 1.1 Grandes étapes de compilation . . . 5
- 1.2 Les étapes de l'analyse . . . 6
- 1.3 Les étapes de la synthèse . . . 7
- 1.4 Le Pipeline de Compilation . . . 9

2 Bases de x86-64 **9**
- 2.1 Modèle Simplifié . . . 9
- 2.2 L'Assembleur . . . 10
- 2.3 La mémoire en x86-64 . . . 11
- 2.4 Programmation en GAS . . . 12
- 2.5 Exercices: Programmation en GAS . . . 16

Aide-mémoire x86-64 **18**

3 Rappels de C **21**
- 3.1 Pointeurs . . . 24
- 3.2 Bibliothèque C . . . 26
- 3.3 Tableaux . . . 27

4 Fonctions, la pile et la libC **29**
4.1 Organisation de la mémoire . . . 29
4.2 Fonctions en assembleur . . . 30
4.3 La convention d'appel . . . 31
4.4 Utiliser la libC . . . 33
4.5 Quelques exercices . . . 36
4.6 Faut-il programmer en assembleur ? . . . 37

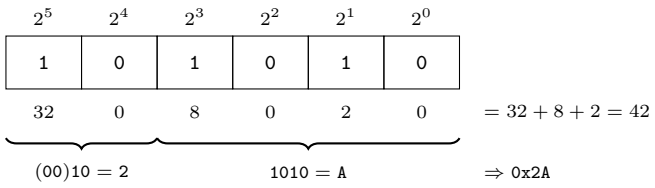
5 Grammaires et analyse syntaxique **37**
5.1 Grammaires formelles . . . 37
5.2 Grammaires non contextuelles . . . 38
5.3 Grammaire ambiguë . . . 39
5.4 Reconnaître une grammaire non contextuelle . . . 41
5.5 Faisons tourner l'algorithme . . . 43
5.6 Utilisation en pratique . . . 44
5.7 Analyse syntaxique descendante . . . 45
5.8 De nombreuses classes... . . . 47

6 Langages réguliers et analyse lexicale **48**
6.1 Langages réguliers . . . 48
6.2 Expressions régulières . . . 48
6.3 Les reconnaître . . . 50
6.4 Automates . . . 50
6.5 Analyse lexicale : le lexeur . . . 51
6.6 Écrire un lexeur . . . 54

0 Prérequis

0.1 Bases de l'informatique

$$\texttt{0b101010} = 1 \times 2^5 + 0 \times 2^4 + 1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 0 \times 2^0 = 32 + 8 + 2 = 42.$$



Hexadecimal uses the digits 0 to 9 and then `A` (= 10) to `F` (= 15), so that

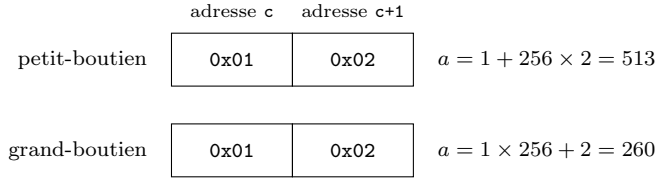
$$\texttt{0x2A} = 2 \times 16^1 + 10 = 32 + 10 = 42.$$

Notation (Prefixes). `0b` marks une représentation binaire and `0x` celle hexadécimale. C uses these same prefixes.

Endianness (petit-boutien / grand-boutien)

Definition (Grand-boutien (*big-endian*) and petit-boutien (*little-endian*))**.** In **grand-boutien** order the octets are written classically with the most significant octet first. In **petit-boutien** order, often when the size of the words is known, the least significant octet comes first: if $m = o_1 o_2$ then

$$m = o_1 + 256\, o_2.$$



Remark. x86-64 is petit-boutien and is used hereinafter.

0.2 Complément à Deux

Negative integers are represented in **complement à deux** (*two's complement*) on k bits.

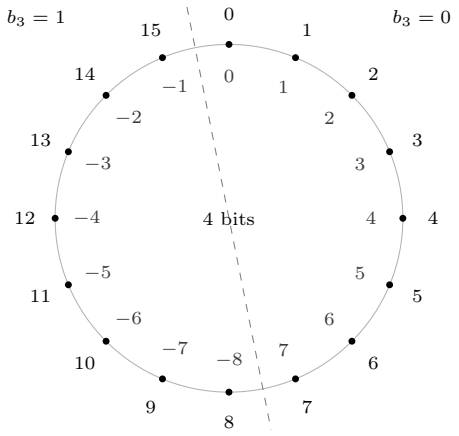
Mathematically, this allows the numbers to live in $\mathbb{Z}/2^k\mathbb{Z}$. The operations $+$, $-$ and $\times$ then work regardless of whether the operands are positive or negative. This does *not* extend to comparisons and to division.

Definition (Complement à Deux)**.** The bit string $b_0 \dots b_k$ represents

$$-2^k b_k + \sum_{i<k} b_i 2^i.$$

The most significant bit is a sign bit: -1 is written `0xFFFF`..., and the sign bit is used to invert the comparisons.

Example (Eight bits)**.** `0x00` $= 0$, `0x7F` $= 127$, `0x80` $= -128$, `0xFF` $= -1$. The last follows from the formula: all bits set gives $-2^7 + (2^7 - 1) = -1$.



outer ring: non signé inner ring: signé (complément à deux)

0.3 Code ASCII

ASCII is a table of characters on 7 bits, i.e. the range $[0 - 127]$. It covers the basic US characters and a set of "control" characters.

Bits b_4	b_3	b_2	b_1	Row / Column	000	001	010	011	100	101	110	111
					0	1	2	3	4	5	6	7
0	0	0	0	0	NUL	DLE	SP	0	@	P	`	p
0	0	0	1	1	SOH	DC1	!	1	A	Q	a	q
0	0	1	0	2	STX	DC2	"	2	B	R	b	r
0	0	1	1	3	ETX	DC3	#	3	C	S	c	s
0	1	0	0	4	EOT	DC4	$	4	D	T	d	t
0	1	0	1	5	ENQ	NAK	%	5	E	U	e	u
0	1	1	0	6	ACK	SYN	&	6	F	V	f	v
0	1	1	1	7	BEL	ETB	'	7	G	W	g	w
1	0	0	0	8	BS	CAN	(	8	H	X	h	x
1	0	0	1	9	HT	EM	)	9	I	Y	i	y
1	0	1	0	10	LF	SUB	*	:	J	Z	j	z
1	0	1	1	11	VT	ESC	+	;	K	[	k	{
1	1	0	0	12	FF	FS	,	<	L	\	l	¦
1	1	0	1	13	CR	GS	—	=	M	]	m	}
1	1	1	0	14	SO	RS	.	>	N	^	n	~
1	1	1	1	15	SI	US	/	?	O	_	o	DEL

0.4 Unités de Mémoire

The definitions used in these notes for memory units are those of the C standard, which are also those of the x86-64 architecture. The table below gives the size of each unit and the number of values it can represent.

	taille	# valeurs
bit	—	2
octet/byte	8 bits	256
mot/word	2 octets	65 536
double	4 octets	$\sim 4 \cdot 10^9$
quad	8 octets	$\sim 18 \cdot 10^{18}$

Type C	mini	courant
`char`	8	8
`short`	16	16
`int`	16	32
`long`	32	64
`long long`	64	64

Remark. *mot* désigne parfois le nombre de bits du processeurs (64 sur les processeurs modernes).

The table on the right gives, for each C type, the width the standard *guarantees* and the width used in common implementations.

0.5 Rappels de C

C'est un langage de programmation qui est **proche de la machine** (*low-level*), **impératif**, avec **passage par valeur** (*pass-by-value*), **statiquement typé**, et l'indentation ne joue aucun rôle (contrairement à Python).

1 Compilateurs

A compilateur (*compiler*) are software systems that translates from the programming languages that humans write into a form in which it can be executed by a computer.

Definition (Compilateur)**.** Un **compilateur** est un programme qui traduit un "programme" d'un langage à un autre en préservant la sémantique.

Example (Compilateurs)**.** ocamlopt, latex, gcc

An interpréteur is another common kind of language processor, which reads a program and executes it directly, without producing a separate output file. The machine-language target program produced by a compilateur is typically much faster, but an interpréteur often gives better error diagnostics.

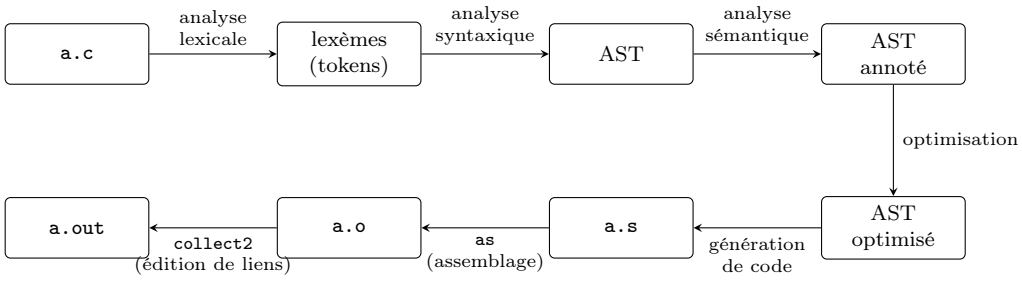
Example (Hybrid systems)**.** Certain language processors systems, such as Java's `javac` and `java`, combine both approaches: `javac` compiles Java source into an intermediate bytecode, which `java` then interprets.

Properties of a compilateur :

- Correct: le code compilé fait ce que l'on attend;
- Fast: le compilateur compile rapidement; and
- Optimising: le compilateur produit un code efficace.

1.1 Grandes étapes de compilation

This mapping between the source program to the target program is done in two main parts: **analysis** and **synthesis**.



The *analysis* stages breaks up the source program into constituent peices and imposes a grammatical structure on them. It then uses this structure to create an intermediate representation of the source program. The *synthesis* stages construct the target program from the intermediate representation and the information in the symbol table.

1.2 Les étapes de l'analyse

Analyse Lexicale

Definition (Analyse lexicale (*lexing*))**.** L'**analyse lexicale** a pour objectif de découper le fichier en une liste de lexèmes (*tokens*); souvent à l'aide d'expressions régulières.

The lexeur is the only stage that sees the file as a sequence of characters. After it, whitespace, indentation and line breaks no longer exist.

Example (The token stream of `fibo`)**.** The source on the left lexes to the stream on the right.

```
int fibo(int n)
{
  if(n<=1)
    return 1;
  return
     fibo(n-1)
   + fibo(n-2);
}
```

```
INT ID("fibo") LPAR INT ID("n") RPAR
LBRACKET
   IF LPAR ID("n") LEQ CST(1) RPAR
     RETURN CST(1) SEMICOL
   RETURN
    ID("fibo") LPAR ID("n") MINUS CST(1) RPAR
   PLUS
    ID("fibo") LPAR ID("n") MINUS CST(2) RPAR
   SEMICOL
RBRACKET
```

Remark. This mapping into the token stream depends on the implementation and the grammar of the source langauge, but each token usually carries an identifier and a value.

Analyse Syntaxique

Definition (Analyse syntaxique (*parsing*))**.** L'**analyse syntaxique** a pour objectif de récupérer la structure du code, généralement sous la forme d'un **arbre de syntaxe abstrait** (*Abstract Syntax Tree*, AST) et souvent à l'aide d'une **grammaire**.

Example (The AST of `fibo`)**.** The token stream on the left parses to the AST on the right.

```
INT ID("fibo") LPAR INT ID("n") RPAR
LBRACKET
   IF LPAR ID("n") LEQ CST(1) RPAR
     RETURN CST(1) SEMICOL
   RETURN
    ID("fibo") LPAR ID("n") MINUS CST(1) RPAR
   PLUS
    ID("fibo") LPAR ID("n") MINUS CST(2) RPAR
   SEMICOL
RBRACKET
```

```
Function( "fibo", [("n",Tint)],
   Sequence(
     If(Less(Var "n", Cst 1),
        Return (Cst 1)),
     Return Plus(
       Call("fibo",[Minus(Var "n", Cst 1)]),
       Call("fibo",[Minus(Var "n", Cst 2)])
      )
   )
)
```

Remark. The AST is only a common form of intermediate representation used by compilers. Alternatives include the *three-address code* and the *control-flow graph*.

Analyse Sémantique

Definition (Analyse sémantique (*semantic analysis*))**.** L'**analyse sémantique** checks for semantic consistency with the language definition, and further gathers type information and saves it.

This stage does not change the shape of the tree, only annotating it and reject semantically inconsistent programs.

Remark. Certain language specifications may permit type conversions known as *coercions*, which typically overloads various operators and functions, and require conversion between types.

Optimisation

Definition (Optimisation)**.** L'**optimisation** a pour objectif de produire un code plus performant.

An optimisation is a rewriting of the program which must preserve the semantics; the analyses that justify the rewriting are what the names below refer to.

Example (Élimination du code mort (*dead code elimination*))**.** The call under a condition that is statically false is removed.

```
int fibo(int n) {                        int fibo(int n) {
  if(n<=1)                                 if(n<=1)
    return 1;                                return 1;
  if(false)                    ==>         return
    printf("fib %d\n",n);                    fibo(n-1)
  return                                   + fibo(n-2);
     fibo(n-1)                           }
   + fibo(n-2);
}
```

Example (Propagation des constantes (*constant propagation*))**.** A product of four `const int`s is evaluated at compile time.

```
const int days_in_year = 365 ;               const int days_in_year = 365 ;
const int hours_in_day = 24 ;                const int hours_in_day = 24 ;
const int min_in_hours = 60 ;          ==>   const int min_in_hours = 60 ;
const int seconds_in_hours = 60 ;            const int seconds_in_hours = 60 ;
int main() {                                 int main() {
  printf("There are %d "                       printf("There are %d "
  "seconds in an ordinary year!",              "seconds in an ordinary year!",
  days_in_year * hours_in_day *                31536000);
  min_in_hours * seconds_in_hours );         }
}
```

Example (Analyse des alias (*alias analysis*))**.** The load is replaced by the value that was just stored.

```
int f(int * a, int * b) {            int f(int * a, int * b) {
  *a = 42;                             *a = 42;
  *b = 12;                  ==>        *b = 12;
  return *a;                           return 42;
}                                    }
```

This rewrite is legal exactly when the compiler can prove that `a` and `b` do not *alias*, i.e. do not point at the same cell.

Optimisation is non-trivial, often requiring global analyses of the program, and is a major part of the work of a compiler. There are also other forms of optimisations, such as dépliement de boucles (*loop unrolling*), analyse de vie (*liveness analysis*), dead store, and sous-expressions communes (*common subexpression elimination*).

1.3 Les étapes de la synthèse

Code Génération

The first stage of *synthesis* is code generation, which turns the intermediate representation into the target language. In the context of *code assembleur (x86)*, this produces a (still) human-readable text file of assembly instructions.

Example (`fibo` in x86-64 AT&T assembly)**.** What `cc1` emits for the source above.

```
    .file   "a.c"
    .text
    .globl  fibo
    .type   fibo, @function
fibo:
```

```
.LFB0:
    .cfi_startproc
    pushq    %rbp
    .cfi_def_cfa_offset 16
    .cfi_offset 6, -16
    movq     %rsp, %rbp
    .cfi_def_cfa_register 6
    pushq    %rbx
    subq     $24, %rsp
    .cfi_offset 3, -24

    ⋮
```

Remark. The lines beginning with a dot are **directives** addressed to the assembler, not instructions: `.text`, `.section .rodata`, `.globl`, `.type` and `.size` organise the output. They produce no machine code.

Assemblage

Definition (Assemblage)**.** In the context of Linux / ELF, the assembleur turns that text into an fichier objet (*object file*). The instructions are now octets, grouped into named sections.

Example (A hex dump of `b.o`)**.** What `as` emits, section by section.

```
b.o:     file format elf64-x86-64

Contents of section .text:
 0000 554889e5 534883ec 18897dec 837dec01  UH..SH....}...}.
 0010 7f07b801 000000eb 1e8b45ec 83e80189  ..........E.....
 0020 c7e80000 000089c3 8b45ec83 e80289c7  .........E......
 0030 e8000000 0001d848 8b5df8c9 c3        .......H.].....
Contents of section .comment:
 0000 00474343 3a202844 65626961 6e203132  .GCC: (Debian 12
 0010 2e322e30 2d31342b 64656231 32753129  .2.0-14+deb12u1)
 0020 2031322e 322e3000                     12.2.0.
Contents of section .eh_frame:
 0000 14000000 00000000 017a5200 01781001  .........zR...x.
 0010 1b0c0708 90010000 1c000000 1c000000  ................
 0020 00000000 3d000000 00410e10 8602430d  ....=....A....C.
 0030 06458303 730c0708                    .E..s...
```

Édition de liens

Definition (Édition de liens (*linking*))**.** In Linux / ELF, the linker takes one or more object files and produces a single executable file. It resolves the references between the object files, and adds the C runtime startup code and the dynamic-linking machinery.

This produce the fichier binaire (*binary file*) that the operating system can load and run.

Example (A word dump inside `a.out`)**.** A window onto the symbol names the linker had to resolve.

```
0000470 5f00 6c5f 6269 5f63 7473 7261 5f74 616d
0000480 6e69 5f00 635f 6178 665f 6e69 6c61 7a69
0000490 0065 7270 6e69 6674 6c00 6269 2e63 6f73
00004a0 362e 4700 494c 4342 325f 322e 352e 4700
00004b0 494c 4342 325f 332e 0034 495f 4d54 645f
00004c0 7265 6765 7369 6574 5472 434d 6f6c 656e
00004d0 6154 6c62 0065 5f5f 6d67 6e6f 735f 6174
00004e0 7472 5f5f 5f00 5449 5f4d 6572 6769 7374
00004f0 7265 4d54 6c43 6e6f 5465 6261 656c 0000
```

Each group is a 16-bit word printed in hexadecimal. On a petit-boutien machine, we may recover the null-terminated strings by swapping each pair of two octets and reading the result as ASCII:

```
__libc_start_main   __cxa_finalize   printf   libc.so.6
GLIBC_2.2.5   GLIBC_2.34   _ITM_deregisterTMCloneTable
__gmon_start__   _ITM_registerTMCloneTable
```

1.4 Le Pipeline de Compilation

Example (Linux / ELF)**.** Invoking `gcc a.c` triggers three tools in sequence:

- `cc1 a.c a.s` — produce the assembly from the C;
- `as -o a.o a.s` — produce the object from the assembly;
- `collect2 a.o a.out` — produce the binary from the object.

Each of these tools performs its own lexical, syntactic and semantic analyses. Additionally, each of the intermediate files can be inspected with the following commands:

- `gcc -S a.c` — produce the assembly from the C;
- `as -o a.o a.s` — produce the object from the assembly;
- `objdump -d a.o` — produce a disassembly of the object;
- `objdump -d a.out` — produce a disassembly of the binary.

The compiler is a chain of programs, each potentially taken as compilers in their own right, with its own input and output language.

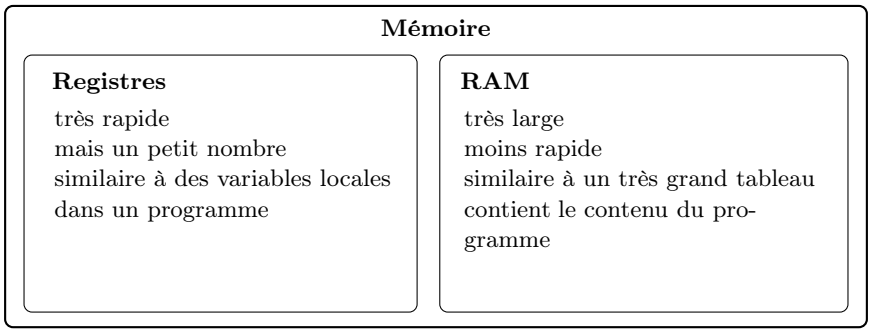
2 Bases de x86-64

A compiler ultimately has to emit machine code, so the bottom of the pipeline has to be understood before the top of it means anything. This chapter is the target language: the x86-64 **instruction set architecture** (*Instruction Set Architecture*, ISA), the GAS syntax in which it is written by hand, and enough of Linux's system-call interface to get a program to produce visible output without any library at all.

2.1 Modèle Simplifié

Le Mémoire d'un Ordinateur

Le modèle d'un ordinateur used in this course has two kinds of storage: **Registres** (*registers*), and **RAM** (*Random Access Memory*).



Le Programme

Definition (Programme)**.** Un **programme** ce sont des instructions dans la RAM à une adresse.

Un registre spécial — `%rip` en x86-64, le **pointeur d'instruction** (*instruction pointer*) — contient l'adresse de l'instruction courante.

Chaque instruction peut:

- lire ou écrire des registres;
- lire ou écrire une case de la RAM; ou
- faire un appel à l'OS (**syscall**).

Remark. Ce modèle ne gère pas : disque, écran, clavier, souris, etc. Tous les effets du programme sur le monde extérieur passent par l'OS.

Example (Factorial of R_0 in the abstract model)**.** With registers R_0, R_1 and R_{IP}, the factorial of R_0 is computed by the following six instructions. Only the writes to R_{IP} make control flow.

Instruction	Effect	
0	$R_1 = 1, \quad R_{IP} = R_{IP} + 1$	R_{IP} = next instruction
1	if $R_0 = 0$ then $R_{IP} = 5$, else $R_{IP} = R_{IP} + 1$	conditional jump
2	$R_1 = R_0 \times R_1, \quad R_{IP} = R_{IP} + 1$	
3	$R_0 = R_0 - 1, \quad R_{IP} = R_{IP} + 1$	
4	$R_{IP} = 1$	unconditional jump
5	rest of the program, or end	

2.2 L'Assembleur

Definition (Assembleur)**. Assembleur** est le langage lisible dans lequel on écrit du code traduit ensuite vers du binaire. Aussi: Le programme qui fait cette traduction.

Instruction Set Architecture

Definition (Instruction Set Architecture (ISA))**. ISA** est l'ensemble des instructions qu'un processeur peut exécuter, et la manière dont elles sont codées en binaire.

The assembleur depends on the ISA but there may be multiple syntaxes for the same ISA.

Example (Intel vs AT&T/GAS syntax)**.** The Intel syntax is used in the Intel manuals and in the Visual Studio IDE. The AT&T/GAS syntax is used by the GNU Assembler (GAS) and by GCC. Between the two syntaxes the order of operands is reversed: Intel writes `mov rax, rbx` while GAS writes `mov %rbx, %rax`.

Le Format Binaire

Definition (Processus)**.** Un **processus** (*process*) est un programme en cours d'exécution. Il peut aussi être vu comme un état de la mémoire: le contenu des registres et de la RAM.

Definition (Exécutable)**.** Un **exécutable** (*executable*) est un fichier qui contient un programme et les informations nécessaires pour que l'OS le charge en mémoire et lancer le processus correspondant.

There are several formats for exécutables:

- **Executable and Linking Format** (ELF) under Unix, except Mac;
- **Portable Executable** under Windows;
- **Mach-O** or PEF under Mac.

Remark. In this course, we study the ELF format, used in Linux, with the x86-64 ISA and the GAS syntax.

2.3 La mémoire en x86-64

Les Registres Généraux

x86-64 exposes sixteen 64-bit registres généraux:

Compute registers		Pointer registers	
Name	"Historical" function	Name	"Historical" function
`rax`	Accumulator	`rsi`	Source index
`rbx`	Base	`rdi`	Destination index
`rcx`	Counter	`rsp`	Stack Pointer
`rdx`	Data	`rbp`	Base Pointer

along with 8 x86-64 registers with no pre-established function, `r8` through `r15`.
Each register can be addressed at four widths by their **sous-registres** (*subregisters*), each a *part of* the same physical register. e.g. Writing `%al` leaves the upper 56 bits of `%rax` untouched.

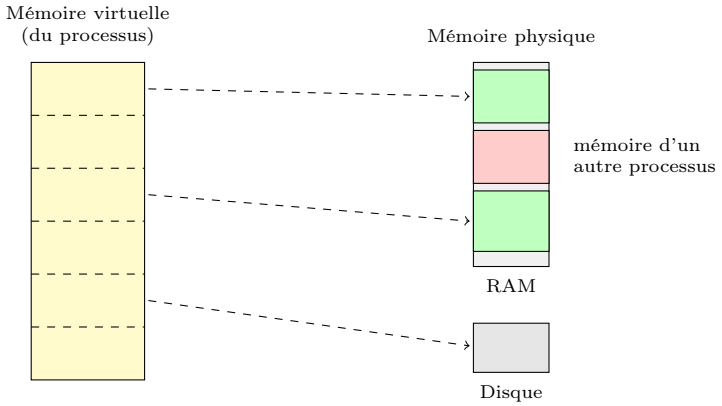
$b_0 \dots b_{63}$	$b_0 \dots b_{31}$	$b_0 \dots b_{15}$	$b_8 \dots b_{15}$	$b_0 \dots b_7$
Quad (q)	Double (d/l)	Word (w)	Byte (b)	Byte
`rax`	`eax`	`ax`	`ah`	`al`
`rbx`	`ebx`	`bx`	`bh`	`bl`
`rcx`	`ecx`	`cx`	`ch`	`cl`
`rdx`	`edx`	`dx`	`dh`	`dl`
`rsi`	`esi`	`si`		`sil`
`rdi`	`edi`	`di`		`dil`
`rsp`	`esp`	`sp`		`spl`
`rbp`	`ebp`	`bp`		`bpl`
`r8`	`r8d`	`r8w`		`r8b`

The `r8` row is the pattern for all eight registers: `r`*n*`d`, `r`*n*`w`, `r`*n*`b`.

Remark. The suffix letters `q`, `l`, `w`, `b` can also be appended to instructions to state how many bytes the operation touches.

Adressage Virtuel

Memory is seen as an array of 2^{64} bytes. Each process has its own translation from the address to memory, which is not necessarily on the RAM. This translation is provided by the OS, depending on its state of paging.



2.4 Programmation en GAS

Définir des sections en GAS Based on the mémoire virtuelle of the processus, we may hence define the sections in which our data lives:

```
        .text # debut d'une section de code
# ce qui est defini ici (donnees ou instructions) est executable

        .data # section de donnees modifiables
# ce qui est defini ici est modifiable mais n'est pas executable
.quad 0x1234567890 # la section commence par
      # 8 octets contenant le nombre 0x1234567890
      # qui prend minimum 5 octets a etre stocke
.byte 0x12 ; ces 8 octets sont suivi par un octet contenant
      ; 0x12 (=18 en decimal)
.ascii "Hello " ; H, e, l, l, o, ' '
.asciz "World" ; 6 octets W, o, r, l, d, \0
.helloworld: ; un label, une maniere d'adresser
        .string        "Hello World!\n" # la meme chose que asciz, donc 13+1=14 octets

        .bss # section de donnees modifiables initialisees a 0

        .text         # une autre section de code
        .globl _start # on veut exporter ce symbole
_start: # le label main correspondra a une function
        movq $42, .x(%rip)
        ret

# GCC rale si on ne finit pas par une speciale section protegee
.section        .note.GNU-stack,"",@progbits
```

Remark. The trailing `.note.GNU-stack` is required for the linker in `gcc` to be able to certify that the program does not want an executable pile.

Registres et constantes, accès à la RAM

In GAS,

- on préfixe les registres par `%` — `%rip`, `%rbx`, etc.;
- on préfixe les nombres par `$` — `$42`, `$0x2A`, etc.

Plus généralement, `offset(base, index, scale)` accède à

$$\text{offset} + v(\text{base}) + v(\text{index}) \times \text{scale},$$

avec *offset* en constante 32 bits signés, *base* un registre, *index* un registre, *scale* un entier parmi $1, 2, 4, 8$. Any of the four parts may be omitted. On peut lire ou écrire 1, 2, 4 ou 8 octets consécutifs.

Operand	Cell addressed
`(%rsp)`	la case à la valeur de `%rsp`
`4(%rsp)`	la case à la valeur de 4+ `%rsp`
`4(%rsp,%rdi,2)`	la case à la valeur de 4+ `%rsp` + 2× `%rdi`
`x(%rip)`	the cell at the label `x` (see below)

Labels et multisections Un registre spécial (%rip en x86-64) stocke l'adresse de l'instruction courante. Labels store addresses relative to %rip, so that the code can be position-independent. We may hence use labels to obtain the address of a stored variable:

```
        .text
.globl _start # marque que le symbole _start sera visible
_start:       # dans le fichier ELF produit
      # differents adressages
      mov x(%rip), %rax # relatif
      lea x(%rip), %rcx # adresse
      ### toujours mettre label(%rip) pour parler de la valeur
      # a la position du label
# on n'est pas oblige de changer de section
# pour stocker notre nombre
x: .quad 0x1234567890
```

Example (Copying Data)**.** La copie de données se fait avec `mov source destination` où `source` et `destination` peuvent être :

- un nombre comme `$42` (décimal) ou `0x2A` en hexadécimal;
- un registre `%rbx`;
- une case mémoire `(%rsp)`, `4(%rsp)`, `18(%rsp,%rax,4)`, `x(%rip)`.

Remark. On peut spécifier le nombre d'octets à copier. It is specified with the suffixes `b`, `w`, `l` and `q` for 1, 2, 4 and 8 bytes, respectively. L'assembleur tente de deviner le nombre d'octets à copier mais ce n'est pas toujours possible, p. ex. `mov $42, (%rsp)`. When the suffix is omitted the guess is made from the operands, and there it fails because neither operand carries a width. Toutes les combinaisons source/destination ne sont pas possibles. Memory to memory copies, for instance, are not allowed, so `mov (%rsp),(%rax)` is illegal.

```
mov $42, %rax
mov helloworld(%rip), %rsi
mov $42, %eax
mov $42, %ax
mov $42, %al
movb $0x12, (%rax) ; ecrit un seul octet (byte)
movw $0x1234, (%rax) ; ecrit un mot (2 O)
movl $0x12345678, (%rax) ; ecrit mot long (4 O)
movq $0x12345678, (%rax) ; ecrit un quad (8 O)
                         ; en utilisant un 32 bits signe
movq $0x4883ec08, 4(%rip)
```

Casting

Extending a value from a smaller width to a larger one is called **casting**. In x86-64, the choice between the two forms is the choice of how the new high bits are filled.

Example (Casting)**.** `movq $0x12345678, (%rax)` writes `0x0000000012345678`, while `movq $0xFFFFFFFF, (%rax)` writes `0xFFFFFFFFFFFFFFFF`, since the immediate is encoded as a signed 32-bit value and sign-extended to 64 bits.

We may hence specify the extension from a source width to a destination width with zero- or sign-extension:

```
movzbl %al, %ecx ; extension with Zero
movswq %ax , %rcx ; extension Signed:
                  ; 1 if starts with 1, 0 otherwise
```

Example (C to Assembly)**.** We then have the following example of a C program and its translation to assembly, obtained with `gcc -S -O0`. Note that an `int` is four bytes, so we use `movl` and store it in a `.long`; the `pushq %rbp` / `movq %rsp, %rbp` / `popq %rbp` trio is the function prologue and epilogue; and the valeur de retour of `main` is stored in `%eax`.

```
int x = 0x12;   // 18
int main() {
  x = 0x34;     // 52
  x = 0x56;     // 86
  x = 0x78;     // 120
}
```

```
        .globl  x
        .data
x:      .long   18
        .text
        .globl  main
main:
        pushq   %rbp
        movq    %rsp, %rbp
        movl    $52, x(%rip)
        movl    $86, x(%rip)
        movl    $120, x(%rip)
        movl    $0, %eax
        popq    %rbp
        ret
```

Example (Disassembly)**.** Below is the disassembly of the previous example. Utilisation de `objdump -S` pour le `main` et `-d` pour le data.

```
0000000000001129 <main>:
  1129: 55                    push %rbp
  112a: 48 89 e5              mov %rsp,%rbp
  112d: c7 05 d9 2e 00 00 34  movl $0x34,0x2ed9(%rip) # 4010 <x>
  1134: 00 00 00
  1137: c7 05 cf 2e 00 00 56  movl $0x56,0x2ecf(%rip) # 4010 <x>
  113e: 00 00 00
  1141: c7 05 c5 2e 00 00 78  movl $0x78,0x2ec5(%rip) # 4010 <x>
  1148: 00 00 00
  114b: b8 00 00 00 00        mov $0x0,%eax
  1150: 5d                    pop %rbp
  1151: c3                    ret
.....
Contents of section .data:
 4000 00000000 00000000 08400000 00000000 .........@......
 4010 12000000                            ....
```

Remark. Instructions are variable-length on x86-64. The three `movl` stores each occupy ten bytes while `ret` occupies one.

Arithmétique

```
add $42, %rax ; %rax += 42
add $-4, %rsp ; %rsp += -4
add %rax, %rbx ; %rbx += %rax
sub %rax, %rbx ; %rbx -= %rax
inc %rbx ; %rbx++
incq (%rsp) ; interpret the 8 byte at positions %rsp
decq (%rsp) ; as an integer and inc/dec them
cmp %rax, %rbx ; compute the diff but stores nothing

imul $42, %rax ; %rax *= 42
imul %rbx ; %rax = (%rax * %rbx) mod 2^64
          ; %rdx = (%rax * %rbx) / 2^64
          ; or %rax:%rdx = (%rax * %rbx)
          ; as signed 128 bits int
mul %rbx ; %rax:%rdx = %rbx * %rax unsigned

idiv %rbx ; %rax = computes %rax + 2^64 x %rdx / %rbx
          ; %rdx is the remainder % %rbx
          ; idivq segfaults if the result does not fit

lea -8(%rsp, %rdx,4), %rax ; %rax = -8+%rsp+%rdx*4
      ; usually used to compute addresses
```

The one-operand forms are implicit in `%rax` and `%rdx`. For division the dividend is the 128-bit quantity `%rdx:%rax`:

$$\texttt{\%rax} := \left\lfloor \frac{2^{64}\,\texttt{\%rdx} + \texttt{\%rax}}{\texttt{\%rbx}} \right\rfloor, \qquad \texttt{\%rdx} := \left(2^{64}\,\texttt{\%rdx} + \texttt{\%rax}\right) \bmod \texttt{\%rbx}.$$

Manipulation Bit-À-Bit

```
not %rax ; reverse all bits
xor %rax, %rbx ; %rbx ^= %rax
or %rax, %rbx ; %rbx |= %rax
and %rax, %rbx ; %rbx &= %rax

sal $42, %rax ; Shift arithmetic left
sar $42, %rax ; Shift arithmetic right
shr $42, %rax ; Shift (logic) right

test %rax, %rbx ; computes & but stores nothing
```

Sauts

```
jmp main ; go to main, do not put (%rip) here
jmp *%rax ; go to the address pointed by %rax
```

A jump target is a label written bare — *not* `main(%rip)`, because the operand of a jump already is an address rather than the contents of one. The starred form `jmp *%rax` is the indirect jump, and it is what function pointeurs, `switch` tables and virtual dispatch all compile to.

Drapeaux Utiles

Il existe un **registre de drapeaux** (*flags register*) spécial, on y retrouve les drapeaux suivants :

Bit	Abréviation	Nom	Description
0	CF	Carry Flag	retenue ? (was there a carry?)
2	PF	Parity Flag	pair ou impair ? (even or odd?)
6	ZF	Zero Flag	nul ? (is it zero?)
7	SF	Sign Flag	strictement négatif ? (strictly negative?)
11	OF	Overflow Flag	le calcul signé a débordé ? (did the signed computation overflow?)

Instructions Conditionnelles

```
jCOND fact ; jump on COND
cmovCOND %rbx, %rcx ; mov on COND
setCOND %rax ; set %rax to 1 when COND
             ; and to 0 otherwise
```

Every condition is invertible by inserting `n`, so `jne` is the negation of `je`, `cmovnz` of `cmovz`, and so on. Quelques conditions usuelles :

`e`	equal	`ae`	above or equal / unsigned
`z`	zero (same as above)	`g`	greater / signed
`ge`	greater equal	`l`	less / signed
`b`	below / unsigned	`ge`	greater or equal / signed
`a`	above / unsigned	`le`	less or equal / signed
`be`	below or equal / unsigned	`p`	parity is even

Interaction avec l'OS

En x86-64, c'est la commande `syscall` qui rend la main à l'OS. The syscall number goes in `%rax` and the arguments in `%rdi`, `%rsi`, `%rdx` (and then `%r10`, `%r8`, `%r9`).

%rax	Nom	%rdi	%rsi	%rdx
0	Read	fd	char*	count
1	Write	fd	char*	count
12	Break	brk		
60	Exit	error		

Remark. En Linux tout est fichier notamment l'entrée, la sortie et la sortie d'erreur avec les **descripteurs de fichiers** (*file descriptors*, `fd`) 0, 1, et 2. On désigne par `fd` 0 la entrée standard, et par `fd` 1 la sortie standard.

2.5 Exercices: Programmation en GAS

Example (A program that does nothing). `.globl _start`

```
_start:
        mov $60, %rax ; id of the syscall
        xor %rdi, %rdi ; a trick to set %rdi to 0
        syscall

        .section .note.GNU-stack ; pour gcc
```

Remark. The `syscall` instruction is required to terminate the program, otherwise the CPU will continue executing off into whatever bytes follow.

Example (Hello World). Pour l'appel `Write` il faut `%rax` = 1; la chaîne a une longueur de 13; et il faut un pointeur vers la chaîne dans `%rsi`. Pour avoir le pointeur, il faut que `"Hello World!\n"` existe dans la mémoire; il faut trouver son adresse et la mettre dans `%rsi`.

```
        .data
helloWorld:
        .string "Hello World!\n"
        .text
        .globl _start
_start:
        lea helloWorld(%rip), %rsi ; load the address of the string
        mov $1, %rdi ; stdout = 1
        mov $13,%rdx ; size of string to be printed
        mov $1, %rax ; id of the syscall
        syscall
        mov $60, %rax
        xor %rdi, %rdi
        syscall
        .section .note.GNU-stack
```

Example (Hello World 10 times). `_start:`

```
        mov $10, %r8
.boucle:
        lea helloWorld(%rip), %rsi
        mov $1,        %rdi
        mov $13,%rdx
        mov $1, %rax
        syscall
        dec %r8
        jnz .boucle
        mov $60, %rax
```

```
xor %rdi, %rdi
syscall
.section .note.GNU-stack
```

On utilise `dec` pour décrémenter et `jnz` pour jump non zero. With these, a counter in `%r8` turns the body into a loop.

Remark. Not all registres d'argument are preserved across a `syscall`, and there is no guarantee that the kernel will not trash them, without knowing the implementation of the syscall.

Autres Exercices

Consigne	Exemple
afficher une ligne de `'X'`	`XXXXXX`
afficher une pyramide de `'O'`	`OO` puis `O`
afficher une chaîne dont on ne connaît pas la longueur	`Hello !`
afficher un caractère stocké dans `%r8` (†)	`'c'` = 99
afficher en décimal petit-boutien l'entier dans `%r8`	`24`
calculer 10! avec une boucle	`3628800`

Aide-mémoire x86-64

GAS / AT&T syntax: `op src, dst`. `M[a]` is the memory at address `a`, `S` a registre or memory operand, and *CC* a condition code from the condition-code table.

Copie de données

`mov src, dst`	`dst = src`; `dst` is never an immediate, and an instruction has at most one memory operand
`movsbl src, %r32`	byte → int, sign-extend
`movzbl src, %r32`	byte → int, zero-fill
`movsXY src, %reg`	sign-extend from size *X* to size *Y* (`b w l q`): `movswq`, `movslq`
`movzXY src, %reg`	zero-extend: `movzbw`, `movzwq`; no `movzlq`, since `movl src, %r32` already zeroes bits 32–63
`lea addr, %reg`	`%reg = addr`, reading no memory; `lea x(%rip), %rsi` is `&x`
`cmovCC S, %reg`	`if (CC) %reg = S`, but `S` is always read, and a 32-bit `%reg` always has bits 32–63 zeroed; never 8 bits

Arithmétique

`add src, dst`	`dst += src`
`sub src, dst`	`dst -= src`
`imul src, %reg`	`%reg *= src`, truncated to the registre: the same bits signed or unsigned; `src` may be `$imm`
`inc dst`	`dst += 1`; CF unchanged
`dec dst`	`dst -= 1`; CF unchanged; `dec` then `jnz` is a counted loop
`neg dst`	`dst = -dst`

Multiplication et division sur 128 bits

`imulq S`	signed `%rdx:%rax = %rax * S`
`mulq S`	unsigned, same effect
`idivq S`	signed `%rax = %rdx:%rax / S`, truncated toward 0, and `%rdx = %rdx:%rax % S`, with the sign of the dividend: C's `/` and `%`
`divq S`	unsigned, same effect
`cqto`	`%rdx:%rax = sign-extend(%rax)`; alias `cqo`; run it before `idivq`; before `divq`, set `%rdx = 0`

`%rdx:%rax` is one 128-bit value, high half first. `S` is a registre or memory, never an immediate; with memory the `q` is compulsory. Division by 0, or a quotient that does not fit, raises the divide error: `SIGFPE` on Linux.

Manipulation bit-à-bit

`and src, dst`	`dst &= src`
`or src, dst`	`dst |= src`
`xor src, dst`	`dst ^= src`; `xor %reg, %reg` zeroes `%reg`
`not dst`	`dst = ~dst`
`sal k, dst`	`dst <<= k`
`shl k, dst`	`dst <<= k`: the same instruction as `sal`
`sar k, dst`	`dst >>= k`, arithmetic: copies of the sign bit shift in
`shr k, dst`	`dst >>= k`, logical: zeros shift in

`k` is `$imm8` or `%cl` — no other registre — or omitted for 1. It is taken mod 64 for `q`, mod 32 otherwise.

Comparaisons et `setCC`

`cmp a, b`	`b - a`: sets the flags, stores nothing; a following `jCC` reads as "`b CC a`"; `b` is never an immediate: `cmp $1, %rax`
`test a, b`	`a & b`: sets the flags, stores nothing
`setCC d8`	`d8 = CC ? 1 : 0`, one byte: any 8-bit registre or a byte of memory

Sauts, instructions conditionnelles

`jmp label`	`%rip = label`; a bare label, never `label(%rip)`
`jmp *%reg`	`%rip = %reg`; `jmp *(%reg)` takes the target from memory
`jCC label`	`if (CC) %rip = label`, else fall through

CC	also	flags	reads as
`e`	`z`	ZF=1	equal, zero
`ne`	`nz`	ZF=0	not equal, non-zero
`s`		SF=1	negative
`ns`		SF=0	not negative
`g`	`nle`	ZF=0 and SF=OF	$>$ signed
`ge`	`nl`	SF=OF	$\geq$ signed
`l`	`nge`	SF≠OF	$<$ signed
`le`	`ng`	ZF=1 or SF≠OF	$\leq$ signed
`a`	`nbe`	CF=0 and ZF=0	$>$ unsigned (*above*)
`ae`	`nb`	CF=0	$\geq$ unsigned
`b`	`nae`	CF=1	$<$ unsigned (*below*)
`be`	`na`	CF=1 or ZF=1	$\leq$ unsigned
`p`	`pe`	PF=1	parity even
`np`	`po`	PF=0	parity odd

An `n` negates a base code. `g`/`l` compare signed, `a`/`b` unsigned: values whose top bits differ order oppositely ($-1 < 1$ signed, $2^{64} - 1 > 1$ unsigned), so the wrong family is a silent bug.

La pile et les appels

`push src`	`%rsp -= 8; M[%rsp] = src` (`pushw`: 2 bytes)
`pop dst`	`dst = M[%rsp]; %rsp += 8`
`call fn`	push the return address — the next instruction's — then `%rip = fn`
`ret`	pop the return address into `%rip`
`leave`	`movq %rbp, %rsp; popq %rbp`
`enter $N, $0`	`pushq %rbp; movq %rsp, %rbp; subq $N, %rsp`

`push` and `pop` change no flags.

Drapeaux

flag	bit	set when
CF	0	retenue — carry out of, or borrow into, the top bit: unsigned overflow
PF	2	the low byte of the result has an even number of 1s
ZF	6	nul — the result is 0
SF	7	the top bit of the result is 1: negative, read as signed
OF	11	débordement — signed overflow

Set by `add`, `sub`, `neg`, `inc`, `dec`, `cmp`, `and`, `or`, `xor`, `test` and the shifts (CF = the last bit out; OF undefined unless the count is 1; a count of 0 changes none); `and`, `or`, `xor` and `test` clear CF and OF, and `inc` and `dec` keep CF. Unchanged by `mov`, `lea`, `not`, `push` and `pop`. Read by `jCC`, `setCC` and `cmovCC`.

Modes d'adressage

Example source operands to `mov`:

mode	operand	value
immediate	`$42`, `$0x2A`	the constant
registre	`%rax`	the registre's contents
direct	`0x4033d0`	`M[0x4033d0]` — no `$`
indirect	`(%rax)`	`M[%rax]`
displacement	`8(%rax)`	`M[%rax + 8]`
scaled index	`D(%rb,%ri,s)`	`M[%rb + D + s*%ri]`
`%rip`-relative	`x(%rip)`	`M[x]` for a label `x`, position-independent; no index

In a scaled index such as `8(%rsp,%rcx,4)`, the scale `s` is 1, 2, 4 or 8; any part may be omitted (`D` = 0, `s` = 1, as in `(,%rcx,4)`), and `%ri` is never `%rsp`. A number `D` in `D(%rip)` means `M[%rip + D]`, `%rip` = the next instruction's address. Each access reads or writes 1, 2, 4 or 8 consecutive bytes. x86-64 is petit-boutien: the low byte sits at the lowest address.

Registres généraux, sous-registres

64 bits	32	16	8 high	8 low	role	sauvegardé par
`%rax`	`%eax`	`%ax`	`%ah`	`%al`	Accumulator: valeur de retour; syscall number and result; implicit in `mulq`, `divq`, `cqto`; `%al` = an upper bound on the vector registres a variadic call uses	l'appelant
`%rbx`	`%ebx`	`%bx`	`%bh`	`%bl`	Base	l'appelé
`%rcx`	`%ecx`	`%cx`	`%ch`	`%cl`	Counter: 4th integer argument; shift count; destroyed by `syscall`	l'appelant
`%rdx`	`%edx`	`%dx`	`%dh`	`%dl`	Data: 3rd integer argument; high half and remainder of `mulq`, `divq`	l'appelant
`%rsi`	`%esi`	`%si`		`%sil`	Source index: 2nd integer argument	l'appelant
`%rdi`	`%edi`	`%di`		`%dil`	Destination index: 1st integer argument	l'appelant
`%rbp`	`%ebp`	`%bp`		`%bpl`	Base pointer: the base of the frame	l'appelé
`%rsp`	`%esp`	`%sp`		`%spl`	Stack pointer: the top of the pile	l'appelé
`%r8`	`%r8d`	`%r8w`		`%r8b`	5th integer argument	l'appelant
`%r9`	`%r9d`	`%r9w`		`%r9b`	6th integer argument	l'appelant
`%r10`	`%r10d`	`%r10w`		`%r10b`	4th syscall argument	l'appelant
`%r11`	`%r11d`	`%r11w`		`%r11b`	destroyed by `syscall`	l'appelant
`%r12`	`%r12d`	`%r12w`		`%r12b`	—	l'appelé
`%r13`	`%r13d`	`%r13w`		`%r13b`	—	l'appelé
`%r14`	`%r14d`	`%r14w`		`%r14b`	—	l'appelé
`%r15`	`%r15d`	`%r15w`		`%r15b`	—	l'appelé

- `%rip`, the pointeur d'instruction, holds the address of the *next* instruction. It is usable only as a base: `label(%rip)` or `D(%rip)`.
- Writing a 32-bit registre zeroes bits 32–63 of the 64-bit one; writing an 8- or 16-bit one leaves the other bits unchanged.
- `%ah`–`%dh` cannot appear in an instruction that needs a REX prefix: one with `%sil`, `%dil`, `%spl`, `%bpl`, `%r8`–`%r15` or a `q` size (`movzbq %ah, %rax` is illegal).

Tailles et suffixes

suffix	name	bytes	C	directive
`b`	octet (*byte*)	1	`char`	`.byte`
`w`	mot (*word*)	2	`short`	`.word`
`l`	mot long (*long, doubleword*)	4	`int`	`.long`
`q`	quad (*quadword*)	8	`long`, pointeur	`.quad`

- For `%r8`–`%r15` the 4-byte name ends in `d` (`%r8d`); the instruction suffix is `l`.
- The suffix may be dropped when a registre operand fixes the size (`%rax` ⇒ `q`, `%eax` ⇒ `l`). It is compulsory when none does: `movq $1, (%rsp)`, `incq (%rsp)`, `cmpb $0, (%rsi)`, `idivq (%rax)`. `push` and `pop` default to 8 bytes.
- With `q`, an immediate is at most 32 bits, sign-extended to 64: `movq $-1, (%rax)` stores all ones. Only `mov $imm, %r64` takes a full 64-bit immediate.

Directives

`.text`	code: executable, not writable
`.data`	modifiable data
`.bss`	modifiable data, initialised to 0
`.section .rodata`	read-only data: `gcc`'s constant chaînes de caractères
`.globl sym`	make `sym` global: `_start` (entry, no libc), `main` (with libc), or a data label
`.byte .word .long .quad v`	emit 1, 2, 4, 8 bytes
`.ascii "s"`	the characters, no NUL
`.asciz "s"`, `.string "s"`	the characters and a NUL
`.type .size .file .ident`	metadata that `gcc -S` emits, like the `.cfi_...` lines; skip it when reading

`.section .note.GNU-stack,"",@progbits` declares a non-executable pile. The linker warns without it; by convention it is the last line.

Syntaxe GAS

- `op src, dst`; Intel syntax writes `op dst, src`.
- Data labels: `x(%rip)`, address `lea x(%rip)`; a bare `x` or `$x` is absolute, and `gcc`'s default PIE édition de liens refuses it.
- `%` before a registre, `$` before an immediate. A bare number is a memory address.
- Numbers: `42`, `0x2A`, `0b101010`. A leading `0` means octal: `$010` is 8.
- `#` starts a comment. `;` separates two statements on one line — it is *not* a comment.
- `name:` defines a label, the address of what follows; `gcc`'s own are `.L2`, `.LC0`. A dot-name without `:` is a directive.
- Chaînes de caractères go in `"..."`, with C escapes.

Appels système

`syscall`: the number in `%rax`, the arguments in `%rdi`, `%rsi`, `%rdx`, `%r10`, `%r8`, `%r9`, the result in `%rax`. It destroys `%rcx` and `%r11` and preserves every other registre.

`%rax`	call	`%rdi`	`%rsi`	`%rdx`
0	`read`	fd	`char *buf`	count
1	`write`	fd	`char *buf`	count
12	`brk`	address		
60	`exit`	status		

Descripteurs de fichiers: 0 stdin, 1 stdout, 2 stderr. `write` takes a pointeur, so a registre's value must be stored to memory first. `exit(0)` is `mov $60, %rax; xor %rdi, %rdi; syscall`. `write(1, s, n)` is `lea s(%rip), %rsi; mov $1, %rdi; mov $n, %rdx; mov $1, %rax; syscall`.

Convention d'appel (System V AMD64)

- **Arguments**: integers and pointeurs in `%rdi`, `%rsi`, `%rdx`, `%rcx`, `%r8`, `%r9`; floating point in `%xmm0`–`%xmm7`, counted separately; then the pile.
- **Valeur de retour**: `%rax`, or `%rdx:%rax` for 128 bits; floating point in `%xmm0`.
- **Sauvegardés par l'appelant**: `%rax`, `%rcx`, `%rdx`, `%rsi`, `%rdi`, `%r8`–`%r11` and every `%xmm`. **Sauvegardés par l'appelé**: `%rbx`, `%rbp`, `%rsp`, `%r12`–`%r15`.
- **L'appelant**: saves the registres sauvegardés par l'appelant it still needs, places the arguments, `call`s, removes its arguments from the pile, and restores.
- **La fonction appelée**: prologue, body, epilogue, `ret`.
- **Prologue**: `pushq %rbp; movq %rsp, %rbp`; `pushq` of any registres sauvegardés par l'appelé; `subq $N, %rsp` (so not `enter`, which subtracts first). **Epilogue**: `leave; ret`, after `addq $N, %rsp` and `popq` of those pushed registres, in reverse order.
- **Frame**: `16(%rbp)` the first argument on the pile, `8(%rbp)` the return address, `0(%rbp)` the appelant's `%rbp`; below, the pushed registres sauvegardés par l'appelé, then the locals.
- **Alignement**: `%rsp` is a multiple of 16 when `call` runs, so it is $\equiv 8 \pmod{16}$ on entry. After `pushq %rbp`, further pushes plus the `subq` must total a multiple of 16.

- **Variadic calls** (`printf`, `scanf`): `%al` is an upper bound on the number of `%xmm` arguments — `movl $0, %eax` when there are none.
- **Zone rouge**: the 128 bytes below `%rsp` may be used without moving it, for data not needed across a call.
- **Entry**: `main` or `_start`. With libc, `main` is a function that libc's startup code calls, returning its status in `%eax` with `ret`. With `gcc -nostdlib`, `_start` has no appelant and ends with the `exit` syscall.

Calling `printf`, following the schéma général:

```
        .data
fmt:    .string "%d\n"
        .text
        .globl main
main:   pushq %rbp              # %rsp now 16-aligned
        movq  %rsp, %rbp
        leaq  fmt(%rip), %rdi   # 1st argument: the format
        movq  $42, %rsi         # 2nd argument
        movl  $0, %eax          # no vector arguments
        call  printf
        movl  $0, %eax          # main returns 0
        leave
        ret
        .section .note.GNU-stack,"",@progbits
```

Chaîne de compilation

`gcc -S a.c`	C to `a.s`, at the default `-O0`; `-fverbose-asm` adds comments
`as -o a.o a.s`	assemble to an ELF fichier objet
`gcc a.s -o a`	assemble and link with libc; entry `main`
`gcc -nostdlib a.s -o a`	no libc, no `crt0`; entry `_start`
`objdump -d a.o`	disassemble
`objdump -S a`	disassembly with the C source interleaved (build with `-g`)
`objdump -s -j .data a`	hex dump of a section's contents

`gcc` runs `cc1` (C to assembly), `as` (to an ELF fichier objet) and `collect2` (édition de liens). ELF is the *Executable and Linkable Format*.

3 Rappels de C

Types en C

Dans le langage C les types contrôlent : la **taille en mémoire** que la donnée occupe, et le **comportement de diverses opérations**. Tout type a une taille que l'on peut récupérer avec `sizeof(type)`.

Remark. Le type n'existe *qu'au moment de la compilation*, il n'y a pas de type dynamique. Nothing in the running program carries a tag saying "I am an `int`". A value is a pile of bytes, and the type is the compilateur's private record of how to read them — which is why a wrong cast is silent and a wrong `printf` format prints garbage instead of raising an error.

Type	Typical size	Holds	Note
`char`	1 octet	a character / a small integer	the unit of `sizeof`
`int`	souvent 4	a decimal integer	not fixed by the standard
`long long`	souvent 8	a wide integer	literal suffix `LL`
`void`	1 mais mal défini	nothing	used as a return type, and as `void *`
`T *`	8 on x86-64	an address	any type of pointeur, whatever `T` is

The word "souvent" is the point: `int` is *souvent 4* and `long long` *souvent 8*, because the sizes are properties of the implementation, not of the language. Only `sizeof(char)` is pinned to 1, since it *defines* the unit.

On peut rajouter des **modificateurs** (*modifiers*) pour changer le comportement. They are added in front of a type: `signed` and `unsigned` change how the same bytes are interpreted and therefore how comparison, division and overflow behave, and `const` marks the object as non-modifiable.

Variable

Definition (Variable)**.** Définir une variable en C c'est **réserver de la mémoire**. Il faut donc le type (pour savoir *combien* de mémoire) et on peut rajouter une valeur initiale.

```
int a ;
char b = 42 ;
long long c, d = factoriel(15);
int e, f=12, g=33, h;
```

On modifie une variable avec `=`, comme `b = 42;` ou `x = y;`. La **durée de vie** (*lifetime*) d'une variable est celle du bloc où elle est définie, sa mémoire peut être utilisée pour autre chose avant ou après. Nothing zeroes it on the way in and nothing preserves it on the way out.

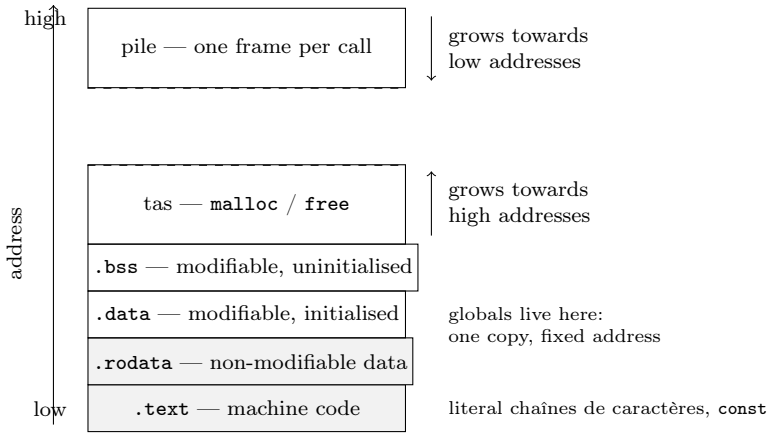
Variables — où sont-elles stockées

This is the section the next chapter is built on. En général, les compilateurs C font les choix suivants :

Situation	Storage	Why
la durée de vie est très courte et le compilateur peut optimiser la variable	un **registre**	pour qu'elle n'existe que dans un registre
la variable est **globale**	un **segment déclaré dans l'assembleur**	il n'existe qu'une version de cette variable (one for the whole program)
la variable est déclarée **dans une fonction**	la **pile**	elle peut exister plusieurs fois (appels récursifs), il faut donc lui allouer de la mémoire pour chaque appel

Which segment a global lands in is decided by two independent bits — is it modifiable, and is it initialised:

Modifiable?	Initialised?	Section
yes	yes	`.data`
yes	no	`.bss`
no	—	`.rodata`



Example (Divers stockages de variables)**.** This example puts one global, one addressed local and one optimised-away local in the same function and shows the assembly the compilateur emits.

```
int globale = 123;

int f() {
  int locale = 456;
  int inlinee = 1+g(&locale);
  return inlinee;
}
```

```
  .text
  .globl f
f:
  subq $24, %rsp
  movl $456, 12(%rsp)    ;;; locale dans 12(%rsp)
  leaq 12(%rsp), %rdi
  call g@PLT
  addl $1, %eax          ;;; inlinee dans %eax
  addq $24, %rsp
  ret
  .globl globale
  .data
globale:                 ;;; globale dans section data car modifiable
  .long 123              ;;; et initialisee
  ;;; dans une section bss si modifiable non initialisee
  ;;; dans une section rodata si non modifiable
```

Reading the three variables off the listing:

Variable	Durée de vie	Where	Evidence
`globale`	the whole program	`.data`	`globale: .long 123`
`locale`	one call to `f`	the pile, at `12(%rsp)`	`movl $456, 12(%rsp)`
`inlinee`	one call to `f`	the registre `%eax`	`addl $1, %eax`

The `subq $24, %rsp` / `addq $24, %rsp` pair is the frame being opened and closed; `locale` occupies four of those twenty-four bytes.

Remark. `locale` and `inlinee` are both locals of the same function and yet they are stored differently. The discriminating fact is in the source code: the program writes `&locale`, so `locale` must have an address, and the compilateur is forced to give it a slot on the pile — hence `leaq 12(%rsp), %rdi`, which hands that address to `g`. `inlinee` is never addressed, is read once, and is therefore allowed to exist only in `%eax`. "Having an address" is exactly the valeur gauche notion of the next subsection, and it is what decides registre versus pile.

Fonctions

Syntaxe générale :

```
type_retour nom_fonction(type1 arg1, type2 arg2) {
   /// code de la fonction
   return valeur_retour;
}
```

Précisions : on met usuellement `type_retour = void` si la fonction ne renvoie rien, et le `return` est facultatif (mais on ne sait alors pas ce que la fonction renvoie).

Example.
```
int produit(int a, int b) { return a*b; }
int life() { return 42; }
void set(int v) { x=v; }
```

Expressions

Les expressions en C sont (entre autres) composées :

- des **constantes** `42`, `0xA2`, `'H'` que l'on peut typer (p. ex. `42LL` est 42 en `long long`);
- des **variables**;
- de **sous-expressions parenthésées**;
- des **appels de fonctions**;
- de diverses **opérations binaires infixes** : `+`, `-`, `*`, `/`, `<`, `>`, `<=`, `>=`, `=`, `==`, `%`;
- de divers **opérateurs unaires** : `!`, `~`, `-`, `*`, `&`, `++`, `--`.

Toute expression a un type qui définit la place nécessaire et le comportement.

Remark. `=` appears in the list of *opérations binaires*, not in a separate list of statements. Assignment is an expression whose value is the value assigned, which is what makes `y=(x=42*f(3,5*g(2)))` legal: the inner assignment to `x` yields a value, and that value is then stored in `y`.

Example (Four expressions)**.**

Expression	What it does
`y=(x=42*f(3,5*g(2)))`	nested calls; the assignment to `x` returns a value
`x=(1+(2<3))`	`2<3` evaluates to 1, so `x` becomes 2
`0.+1/2`	`1/2` is *integer* division, i.e. 0; the result is 0.0
`(0.+1)/2`	`0.+1` is a `double`, so the division is real: 0.5

The last two differ only by a pair of parentheses and by a factor of infinity in the answer, because the type of a sub-expression — not the type of the variable being assigned — decides which division is compiled.

Structures de contrôle

```
while(condition) fairequelquechose

if(condition) fairequelquechose
if(condition) fairequelquechose else fairequelquechose
```

Si `fairequelquechose` est composé de plusieurs instructions il faut les mettre entre accolades. Une condition est **vraie** si en l'évaluant on obtient une valeur *différente de* 0. There is no separate boolean type to satisfy.

Example (Collatz)**.**
```
int n = 0; while(n<42) n++;
while(n!=1) {
  if(n%2)        /// teste la parite car n%2=0 <==> n est pair
    n = 3*n+1 ;  /// n est forcement pair maintenant
  n /= 2;
}
```

The condition `n%2` is used directly as the test: it is 0 when `n` is even and 1 when it is odd, and no comparison against a boolean is written. The comment records the loop's invariant — after the `if` body, `n` is necessarily even, so the unconditional `n /= 2` is exact.

3.1 Pointeurs

Valeur gauche

Definition (Valeur gauche)**.** Une **valeur gauche** (*left-value*) c'est une valeur qui peut être à gauche dans un `=`. En C une valeur gauche c'est une valeur qui est **stockée quelque part en mémoire** (et qui a un type). Autrement dit, une valeur gauche c'est quelque chose qui **a une adresse en mémoire**.

```
int a;
a = 42 ; /// autorise
42 = a ; /// interdit, on ne peut pas modifier 42
```

`a` est une valeur gauche mais pas `42`. There is no memory cell called "`42`" to write into.

Pointeur

Definition (Pointeur)**.** Un **pointeur** (*pointer*) est un objet qui stocke l'adresse mémoire d'un certain type :

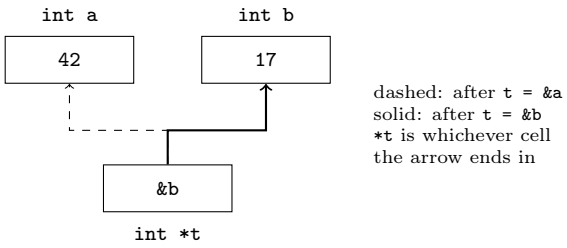
- `T* p` déclare un pointeur `p` vers un objet de type `T`;
- `&a` récupère l'adresse d'un objet `a`;
- `*p` renvoie l'objet de type `T` pointé par `p`;
- `*p` est une valeur gauche car `*p` est stocké à l'adresse `p`.

Remark. `&` is the *address-of* or *referencing* operator and `*` is the *dereferencing* or *indirection* operator: taking `&a` is référencer, evaluating `*p` is déréférencer. The operators themselves are unambiguous — `&` goes from an object to its address, `*` goes from an address to the object — and the two names are easy to swap, so read the symbols and not the names.

```
int a, b ;
int * t ;  /// int* c'est un type de pointeur vers un int
t = &a ;   /// t stocke maintenant l'adresse de a
*t = 42 ;
t = &b ; *t = 17;  /// a vaut 42 et b vaut 17
```

Two identities follow directly from the definition, and they are the ones to recite in an exam.

Proposition (Allers-retours)**.** *Quand* `a` *est une valeur gauche, alors*



`*(&a) == a`,

car `&a` *récupère l'adresse de* `a` *et* `*` *récupère ensuite la valeur à cette adresse. De même si* `a` *pointe vers quelque chose alors*

`&(*a) == a`,

car `*a` *crée une valeur gauche stockée à l'adresse* `a` *et* `&` *récupère ensuite l'adresse de la valeur gauche.*

Remark. `*a` est **toujours** une valeur gauche, `&a` ne l'est **jamais**. The asymmetry is the whole content of the previous subsection: `*a` names a cell, whereas `&a` is a computed address that is not itself stored anywhere. Hence `*p = 42` compiles and `&a = 42` does not.

Arithmétique de pointeurs

Le type des pointeurs change le comportement de l'arithmétique. For `p`, `p1`, `p2` of type `T*` and `i` an integer:

$$\begin{aligned} p + i &\iff p + \texttt{sizeof}(T) \times i, \\ p - i &\iff p - \texttt{sizeof}(T) \times i, \\ p_1 - p_2 &\iff (p_1 - p_2) \,/\, \texttt{sizeof}(T) \qquad \text{for } p_1, p_2 \text{ of the same type } \texttt{T*}, \\ \texttt{p++} &\iff p = p + 1. \end{aligned}$$

Les opérations qui n'ont pas de sens échouent : $p_1 + p_2$, $p_1 * p_2$, etc. Adding two addresses names nothing, whereas subtracting two addresses of the same type names a count of elements.

Remark. The scaling is silent. Si `t` pointe vers un entier 4 octets, `t+1` vers l'entier *d'après* qui est 4 octets plus loin. The source code reads `t+1` and the machine computes $\texttt{t} + 4$. This is also why `void *` arithmetic is ill-defined: `sizeof(void)` is "1 mais mal défini".

Tableaux

Les tableaux sont **presque que des pointeurs** :

```
int t[5] = {1,2,3,4,5} ;
int * x = t;
*x = 1 ;       /// modifie t[0]
*(x+2) = 2 ;   /// modifie t[2]
```

Proposition (L'indexation est du sucre syntaxique)**.** *En C,* `t[i]` *est du* ***sucre syntaxique*** *(syntactic sugar) pour*

`*(t+i)`.

Remark. `+` is commutative, so `*(t+i)` is `*(i+t)`. On peut d'ailleurs écrire `4[t]` plutôt que `t[4]`. It compiles. The bracket is therefore not a primitive operation on a special "array" object but a règle de réécriture on a pointeur and an integer.

L'initialisation de la forme `int t[10]` marche quand on connaît la taille du tableau. Cela réserve $10 \times$ `sizeof(int)` octets. Si on ne connaît *pas* la taille du tableau à la compilation il faut demander de la mémoire à l'OS. Il y a une fonction dans la libC qui gère l'allocation mémoire : `malloc`.

```
int * t = malloc(40) ;
  /// 40 octets, soit 10 int de taille 4

t = malloc( 10 * sizeof(int) ) ; /// autre maniere
/// de demander la meme chose
```

Remark. The two calls request the same thing, but only the second survives a change of architecture: the literal `40` silently assumes `sizeof(int) == 4`, which the table of types above gives as only *souvent* true. Write the `sizeof` form.

Chaînes de caractères

En C, les caractères sont de type `char` et les **chaînes de caractères** (*strings*) sont simplement des tableaux de caractères, et des pointeurs `char*`. Une chaîne de la forme `"machaine"` dans le programme C est une valeur de type `const char *`.

Notation. Par convention, une chaîne de caractères se termine par le caractère `'\0'` dont la valeur numérique est 0. Nothing else records the length: the terminator *is* the length.

```
const char * s = "Hello World!"; /// C rajoute un '\0'
int len = 0 ;
while(s[len])  /// s'arrete quand s[len] vaut 0 = '\0'
  len++;
s[2] = '?' ;   /// que va faire cette ligne ?
```

Remark. The loop needs no comparison: `s[len]` is itself the condition, and it is false exactly on the terminator, since a condition is true when it differs from 0 and `'\0'` *is* 0. On exit, `len` is the number of characters before the terminator.

Remark. The listing's last line is left as a question. Two independent things are wrong with `s[2] = '?'`. The type of `s` is `const char *`, so the pointed-to object is declared non-modifiable and the compilateur objects. And by the storage rules above, non-modifiable data goes in `.rodata`, a section the loader maps read-only — so forcing the write past the compilateur gets it refused by the hardware at run time rather than executed.

3.2 Bibliothèque C

Definition (libC)**.** La **libC** (pour C library et donc bibliothèque C) est un ensemble de fonctions qui permettent à C d'interagir avec l'OS *quel que soit* l'OS. It is the portability layer: the C program calls libC, and libC calls whichever system interface the platform actually has.

La libC contient beaucoup de fonctions, vous devez en connaître 4 :

Function	Role
`malloc`	alloue de la mémoire (déjà vue)
`free`	libère de la mémoire allouée par `malloc`
`printf`	affiche sur la sortie standard
`scanf`	lit sur l'entrée standard

printf

La fonction `printf` prend un nombre arbitraire de paramètres et le premier paramètre est un **format** de type chaîne de caractères. The format is copied to the output verbatim except at the `%` placeholders, each of which consumes one further argument.

Example.
```
printf("Hello World!");
printf("%d",42);
printf("%s","Hello World");
printf("a=%d / b=%d / chaine %s", 1, 2, "coucou");
```

The last line is the general case: several arguments interleaved with literal text, consumed left to right in the order the placeholders appear.

Formats utiles Voici les formats qui peuvent être utiles dans ce cours :

Format	Argument	Size of the argument
`%d`	entier décimal sous forme de `int`	souvent 4 octets
`%lld`	entier décimal sous forme de `long long`	8 octets
`%s`	chaîne de caractères (pointeur vers le premier caractère)	8 octets
`%c`	affiche un unique caractère	—
`%p`	affiche l'adresse d'un pointeur	8 octets

Remark. **Attention** : pour `printf` il faut faire attention à ce que *les tailles des types soient les bonnes*. `printf` is variadic: it cannot inspect what it was handed, so it reads as many bytes off the argument area as the format tells it to. Passing a `long long` to `%d` or an `int` to `%lld` makes `printf` read the wrong number of bytes for that placeholder and print garbage; on ABIs that pass variadic arguments on the pile it additionally shifts every later argument. The compilateur has no type to check against. This is the practical face of "le type n'existe qu'au moment de la compilation".

scanf

La fonction `scanf` est similaire à `printf` excepté qu'elle *lit*. Les arguments ne sont pas tout à fait les mêmes car `printf` prend des **valeurs**, `scanf` doit prendre des **pointeurs** :

- pour lire une chaîne de caractères, ça ne change pas le type, on passe un pointeur où l'on veut que `scanf` stocke la chaîne (comme `printf`);
- pour lire un entier, on passe un pointeur où stocker l'entier.

```
char chaine_lue[100] ;
int entier_lu ;
scanf("%s %d",chaine_lue, &entier_lu);
/// en lisant une chaine avec %s scanf s'arrete a la premiere espace ' ',
/// tabulation '\t' ou retour a la ligne '\n'
/// avec %d l'entier doit etre decimal (%i pour octal ou hexadecimal)
```

Remark. `chaine_lue` is passed bare and `entier_lu` is passed as `&entier_lu`. That is not an inconsistency: the array name already *is* the address of its first element, so it is already the pointeur `scanf` wants, whereas `entier_lu` is an `int` and needs the `&`. Reading `t[i]` as `*(t+i)` makes this automatic.

Remark. Two behaviours of the format worth keeping. First, `%s` s'arrête à la première espace `' '`, tabulation `'\t'` ou retour à la ligne `'\n'`, so it reads one word and not one line. Second, avec `%d` l'entier doit être *décimal* (`%i` pour octal ou hexadécimal, as well as decimal).

3.3 Tableaux

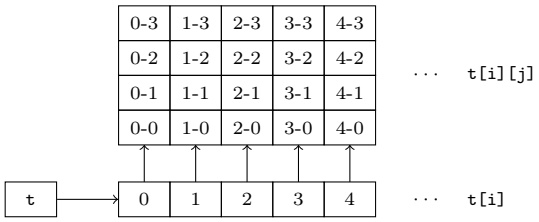
Tableaux 2D

Une solution avec `malloc` : an array of pointeurs to arrays.

```
int ** t ;
/// un pointeur vers des
/// pointeurs vers des entiers

t = malloc( sizeof(int*) * 10 ) ;
/// attention sizeof(int*) !

int i = 0;
while(i<10) {
  t[i] = malloc( sizeof(int) * 10 ) ;
  i++;
}
```



Remark. Attention `sizeof(int*)` in the first `malloc`: the outer allocation holds *pointeurs*, 8 bytes each, not integers. Writing `sizeof(int)` there would reserve half the memory needed and corrupt the spine of the structure.

Remark. C'est le même fonctionnement en OCaml et en Python : un tableau multidimensionnel est un **tableau de tableaux**. The rows are separate allocations, so they need not be contiguous with one another and need not even have the same length.

Tableaux linéarisés

The other solution is the built-in multidimensional array, which is not an array of arrays at all.

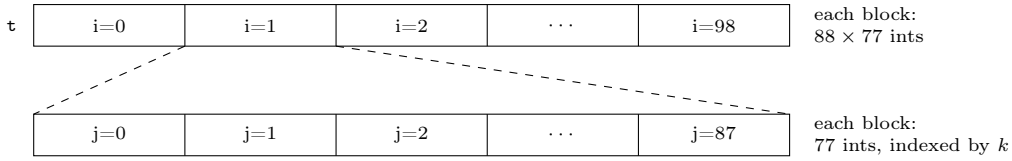
```
int t[99][88][77] ; /// en fait c'est un tableau de taille 99x88x77

int i =3, j=4, k=5;
t[i][j][k] /// est remplace par *(t+i*88*77+j*77+k)
```

A single block of $99 \times 88 \times 77$ integers is reserved, and the indices are folded into one offset. The compilateur turns

$$\texttt{t[i][j][k]} \longrightarrow \texttt{*(t + i*88*77 + j*77 + k)},$$

where the offset is counted in *elements* — the scaling by `sizeof(int)` is then supplied by the arithmétique de pointeurs of the previous subsection. Note which dimensions appear: the first, 99, does not, while 88 and 77 do.



Remark. Le type tableau multidimensionnel est un pointeur avec l'information additionnelle des dimensions (sauf la première) et un peu compliqué à comprendre. Il vaut mieux éviter de faire des choses compliquées avec. The first dimension is absent because it is never needed to compute an offset — only the strides below a given index matter.

Example (Passing a slice)**.**
```
void f(int a[][77]) { a[2][3] = 42 ; } /// a est un pointeur
int (*x)[88][77] = t;
f(x[1]); /// calcule x+1*sizeof(int[88][77]) = 88*77*4
```

The parameter `int a[][77]` is a pointeur that remembers the stride 77, which is all `a[2][3]` needs: the offset is $2 \times 77 + 3$ elements. The variable `x` is a pointeur to an `int[88][77]`, so `x[1]` advances by one whole 88×77 slab — the comment records the arithmetic as $88 \times 77 \times 4$ bytes, the factor 4 being `sizeof(int)`.

Conclusion. The two constructions have the same indexing syntax and nothing else in common. `int **t` is one allocation of pointeurs plus one allocation per row: `t[i][j]` costs two memory reads, the rows are independent, and `free` must be called once per row plus once for the spine. `int t[99][88][77]` is a single contiguous block: `t[i][j][k]` costs one multiply-add and one read, nothing is separately allocated, and the dimensions are baked into the type. Knowing which one is in front of you is the difference between an offset computation and a chase from pointeur to pointeur.

4 Fonctions, la pile et la libC

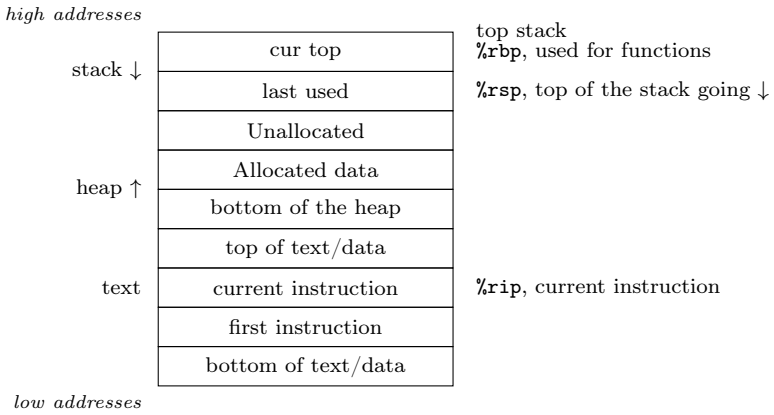
The machine has no notion of a function. It has registres, a flat array of bytes and a program counter, and that is all. Everything that makes a function — parameters, local variables, a valeur de retour, the guarantee that the appelant's data survives the call — is an agreement between two pieces of code that never meet. This chapter states that agreement. It begins with the memory a processus is given, narrows to the region that agreement lives in (the pile), then fixes the System V calling convention that `gcc` and the libC both obey, and ends by calling `scanf` and `printf` from bare assembly.

4.1 Organisation de la mémoire

Definition (Les trois parties de la mémoire d'un programme)**.** En simplifiant beaucoup, la mémoire d'un programme est composée de trois parties :

- la **mémoire statique** (*static memory*) chargée par l'OS depuis l'ELF (le contenu des sections `.text`, `.data`, `.bss`, etc.) ;
- la **pile** (*stack*), qui est une mémoire fournie par l'OS *pour les fonctions* ; l'OS fournit directement cette mémoire ;
- le **tas** (*heap*), qui est une mémoire fournie par l'OS *sur demande*.

The distinction that matters for this chapter is the second one. The mémoire statique is fixed at the édition de liens and the tas has to be requested (through `malloc`, i.e. ultimately through a system call); the pile is simply *there* when the processus starts, without being asked for, which is why a function can allocate its working space in a single subtraction and free it in a single addition.



Read the picture from the bottom: the text and data loaded from the ELF sit at the low addresses, the tas grows *upwards* from just above them, and the pile grows *downwards* from the top. Between the two lies unallocated address space, which is what lets both grow. Three registres point into this picture: `%rip` into the text at the current instruction, `%rsp` at the top of the pile, and `%rbp` somewhat higher up in the pile, "used for functions".

Organisation de la pile pour les fonctions

Definition (Organisation de la pile pour les fonctions (*stack discipline*))**.** Il est *standard* — not enforced by the hardware — que :

- **presque toujours**, `%rsp` pointe vers le premier octet utilisé de la pile ;
- **souvent** `%rbp` pointe vers le début de la pile au début de l'appel de la fonction courante.

Remark. The hedging is worth keeping. "Presque toujours" and "souvent" are the right words because nothing in the processor checks either claim: `%rsp` and `%rbp` are ordinary registres généraux. Only the instructions `push`, `pop`, `call` and `ret` — and the interrupt machinery — treat `%rsp` specially. `%rbp` is special to nobody at all; it is a convention kept by compilateurs.

Manipuler la pile Les commandes suivantes permettent de manipuler la pile :

- `push %rbx` ou `pushq $42` décale `%rsp` de la bonne taille puis stocke `%rbx`/`42` dans `(%rsp)` ;
- `pop %rbx` récupère la valeur `(%rsp)` dans `%rbx` puis décale `%rsp` de la bonne taille.

Ces commandes peuvent s'écrire autrement (`add` + `mov`). Since the pile grows towards the low addresses, "décale `%rsp` de la bonne taille" means *subtracting* the operand size on a push and *adding* it on a pop:

```
pushq   %rbx            # is the pair
subq    $8, %rsp
movq    %rbx, (%rsp)

popq    %rbx            # is the pair
movq    (%rsp), %rbx
addq    $8, %rsp
```

Remark. Note the order inside each pair: a push decrements *then* writes, a pop reads *then* increments. `(%rsp)` therefore always denotes a byte that is in use, never free space. Note also that the two halves are ordinary instructions, so nothing stops you from moving `%rsp` by hand: `subq $32, %rsp` allocates thirty-two bytes in one stroke, which is exactly what a function prologue does, and is far cheaper than four pushes.

4.2 Fonctions en assembleur

Definition (Les fonctions ne sont que des conventions)**.** *En assembleur les fonctions ne sont que des conventions.* In assembly a function is not a language construct; it is a pair of promises, one kept by the code that calls, the **appelant** (*caller*), and one kept by the code that is called, the **fonction appelée** (*callee*), about where arguments live, where the result is left, which registres survive, and where execution resumes.

L'appelant

Avant d'appeler une fonction, `gcc` va :

- sauvegarder les registres que la fonction appelée risque d'écraser ;
- stocker les arguments à des endroits prédéfinis (p. ex. sur les registres puis la pile) ;
- stocker l'adresse de l'instruction à exécuter une fois que la fonction termine ;
- sauter au début de la fonction ;
- en revenant, retirer les arguments mis sur la pile ;
- restaurer la valeur des registres sauvegardés.

The third and fourth steps together are precisely what the `call` instruction does; the rest is code the compilateur emits around it.

La fonction appelée

Pour une fonction appelée, `gcc` va :

- allouer une **frame** et sauvegarder le `%rbp` précédent ;
- faire le calcul ;
- libérer la frame ;
- revenir à l'appelant.

The first item is the **prologue**, the last two the **epilogue**.

Quelques commandes utiles pour les fonctions

instruction	effect
`call label`	plus ou moins équivalent à `push %rip+8` combiné à `jmp label`
`ret`	plus ou moins équivalent à `pop %rip`
`enter 32`	équivalent à `pushq %rbp`, `movq %rsp, %rbp`, `subq $32, %rsp`, mais n'est généralement pas utilisé
`leave`	équivalent à `movq %rbp, %rsp`, `pop %rbp`, un peu plus souvent utilisé

Remark. The "plus ou moins" in the first two rows is essential. `%rip` is not an addressable registre général on x86-64: `push %rip` and `pop %rip` are not instructions you can assemble. The equivalence describes the *effect* — `call` pushes the return address and transfers control, `ret` pops it back into the program counter — not a rewriting you may type. The `+8` likewise stands for "the length of the `call` instruction"; what is pushed is the address of the instruction that follows the `call`.

Remark. `enter` and `leave` are single instructions that package a whole prologue and epilogue. `enter` is slow enough on modern processors that compilateurs avoid it; `leave` survives because it is one byte and does exactly the right thing.

La frame

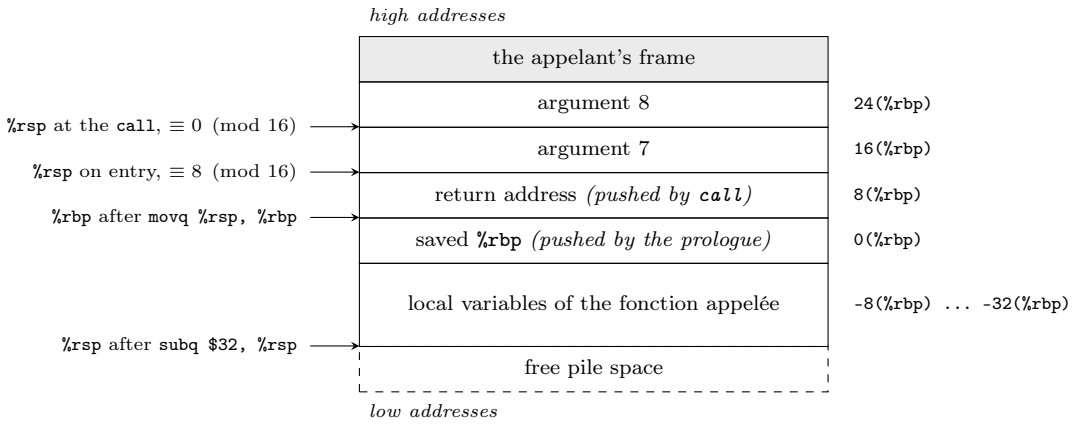
The standard prologue and epilogue, written out longhand, are

```
f:      pushq   %rbp            # save the caller's frame pointer
        movq    %rsp, %rbp      # this frame starts here
        subq    $32, %rsp       # 32 bytes of local variables
        # ... the computation ...
        movq    %rbp, %rsp      # free the frame  (these two lines = leave)
        popq    %rbp
        ret
```

so that `enter 32` replaces the first three lines and `leave` replaces the two that free the frame.
Once `%rbp` has been set, every interesting slot has a fixed offset from it, and those offsets stay valid no matter how much `%rsp` subsequently moves. That is the whole point of keeping a pointeur to the frame: `0(%rbp)` holds the appelant's saved `%rbp`, `8(%rbp)` the return address, `16(%rbp)` the seventh argument if there is one, and the local variables live at the negative offsets `-8(%rbp)`, `-16(%rbp)` and so on. Addressing the same locals off `%rsp` would require recomputing every offset after each push.

4.3 La convention d'appel

The "endroits prédéfinis" of the appelant's second step are not a free choice. On Linux they are fixed by the **System V AMD64 ABI**, which is the convention `gcc` emits and the libC expects. Assembly that calls the libC must obey it exactly; assembly that only calls itself may do as it pleases, but there is no reason to invent a second convention. Naming the ABI and its registres goes beyond this course, which says only that the arguments are stored *à des endroits prédéfinis* and that one must make *des appels alignés sur des multiples de 16*. This subsection fills those two phrases in from the ABI itself.



Où vont les arguments

Definition (Registres d'argument (*argument registers*), dans l'ordre)**.** Arguments that are integers or pointeurs are passed in the registres

`%rdi`, `%rsi`, `%rdx`, `%rcx`, `%r8`, `%r9`

in that order; the seventh and later integer arguments are pushed on the pile. Floating-point arguments go in `%xmm0` through `%xmm7`. The two sequences are consumed independently.

argument n	1	2	3	4	5	6	7	8
integer / pointeur	`%rdi`	`%rsi`	`%rdx`	`%rcx`	`%r8`	`%r9`	pile	pile
floating point	`%xmm0`	`%xmm1`	`%xmm2`	`%xmm3`	`%xmm4`	`%xmm5`	`%xmm6`	`%xmm7`

The arguments passed on the pile are laid out so that the seventh sits at the *lowest* address, that is, at `(%rsp)` at the instant the `call` executes, the eighth at `8(%rsp)`, and so on. They are the appelant's property: removing them after the call is the appelant's fifth step, and it is one addition to `%rsp`.

Definition (Valeur de retour (*return value*))**.** The result comes back in `%rax` (in the pair `%rdx`:`%rax` for a 128-bit value) for integers and pointeurs, and in `%xmm0` for floating point.

Definition (`%al` et fonctions variadiques)**.** For a **fonction variadique** (*variadic function*) — one declared with `...`, such as `printf` and `scanf` — the appelant must additionally set `%al` to the number of vector (`%xmm`) registres used to pass arguments.

This is what the line `mov $0, %rax` does in every libC example below. It is not decoration and it is not a cleared valeur de retour: it announces "no floating-point argument was passed". A fonction variadique uses the count to decide whether it must spill the eight `%xmm` registres into its save area for registres; leaving `%rax` holding garbage makes `printf` read vector registres it was never given, and the call may fault.

Questions d'alignement

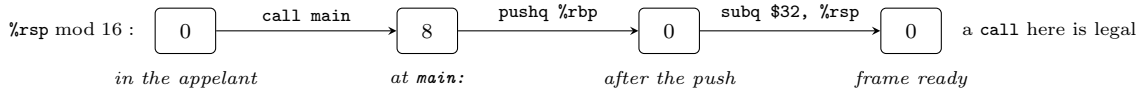
Definition (Alignement (*alignment*) naturel)**.** Quand on lit un bloc de taille 2^k, le processeur s'attend à ce que l'adresse lue soit un multiple de 2^k.

Pour `movq`, `movl`, `movw`, `movb` et toutes les instructions vues jusqu'à présent ce n'est pas un problème, mais il existe des instructions qui *s'attendent* à des adresses alignées. x86 tolerates misaligned ordinary loads and stores, at a speed penalty; the instructions that expect aligned addresses fault otherwise. Même si vos instructions ignorent l'alignement, la libC peut l'attendre, il faut donc faire des appels alignés sur des multiples de 16.

Definition (La règle des 16 octets avant un `call`)**.** At the instant the `call` instruction executes, `%rsp` must be a multiple of 16. Since `call` then pushes an eight-byte return address, on entry to the fonction appelée

$$\texttt{\%rsp} \equiv 8 \pmod{16},$$

and the fonction appelée's own `pushq %rbp` brings it back to a multiple of 16.



Remark. Two corollaries that the exercises trip over. First, the frame size must be a multiple of 16: after the entry offset of 8 and the eight-byte `pushq %rbp`, `subq $32, %rsp` preserves the alignement because $32 \equiv 0 \pmod{16}$, whereas `subq $24, %rsp` would destroy it and the next `call printf` would be misaligned. This is the meaning of the warning *"attention gcc peut attendre un alignement 16 octets"* in the libC skeleton below. Second, an odd number of pushes leaves the pile misaligned, so a function that pushes three registres sauvegardés par l'appelé must push a fourth value — or subtract eight — before it may call anything.

Remark. The ABI also reserves the 128 bytes immediately below `%rsp` as the **zone rouge** (*red zone*), which a leaf function may scribble on without moving `%rsp` at all. It goes beyond this course, and it is unusable in any function that makes a call, since the `call` itself overwrites the top of it.

Registres sauvegardés par l'appelant et par l'appelé

The appelant's first and last steps — *sauvegarder les registres que la fonction appelée risque d'écraser*, then *restaurer la valeur des registres sauvegardés* — raise the obvious question of which registres those are. The convention answers it by splitting the set of registres in two.

sauvegardés par l'appelant, volatile	`%rax`, `%rcx`, `%rdx`, `%rsi`, `%rdi`, `%r8`–`%r11`, and all the `%xmm` registres
sauvegardés par l'appelé, non-volatile	`%rbx`, `%rbp`, `%rsp`, `%r12`–`%r15`

Definition (Les deux promesses)**.** A registre **sauvegardé par l'appelant** (*caller-saved*) may be destroyed freely by the fonction appelée; if the appelant still needs its value after the call, the appelant must save it. A registre **sauvegardé par l'appelé** (*callee-saved*) must have the same value when the function returns as it had on entry; if the fonction appelée wants to use it, the fonction appelée must push it in its prologue and pop it in its epilogue.

Remark. No registre is intrinsically "safe". The split only says *who pays*. It is chosen so that the registres d'argument and the scratch registres are volatile — an appelant that has already consumed its arguments loses nothing by them being clobbered — while a fonction appelée that wants a long-lived value has `%rbx` and `%r12`–`%r15` available at the price of one push each.

Remark. The fonction appelée's first step, "allouer une frame et sauvegarder le `%rbp` précédent", is nothing other than `%rbp` being sauvegardé par l'appelé, and `%rsp` appears in the sauvegardés par l'appelé row for the same reason: the epilogue must hand `%rsp` back exactly where it found it, or `ret` will pop the wrong word.

4.4 Utiliser la libC

Schéma général

```
.data

.text
```

```
        .globl main
main:   ; main est une fonction, on doit ret
        push %rbp
        movq %rsp, %rbp
        subq $32, %rsp ; attention gcc peut attendre
        ; un alignement 16 octets

        ret
        .section .note.GNU-stack
```

Three things in this skeleton are not optional. The global symbol is `main` and not `_start`: the édition de liens against the libC brings in its startup code, which runs first, initialises the library and then *calls* `main`. `main` is therefore an ordinary function and must `ret` — as the comment in the listing says, *main est une fonction, on doit ret* — returning its exit status in `%eax`. Falling off the end, or ending with an `exit` system call, would skip the library's own shutdown.

Remark. The final line `.section .note.GNU-stack` emits the empty note section by which a fichier objet declares that it does not need an executable pile. It carries no code; without it the linker assumes the worst and warns.

Remark. The frame here is thirty-two bytes and the comment on the `subq` flags exactly the trap of the previous subsection: a size that is not a multiple of sixteen leaves `%rsp` misaligned for every subsequent call into the libC.

Remark. The skeleton is nevertheless incomplete as printed: it allocates the frame and never frees it. Between `subq $32, %rsp` and `ret` it has no `leave` and no `movq %rbp, %rsp` followed by `popq %rbp`, so at the `ret` `%rsp` still sits thirty-two bytes below the saved `%rbp`: `ret` pops a local variable rather than the return address, and control jumps to garbage. The blank line is where the body goes, and the epilogue written longhand above — `movq %rbp, %rsp` then `popq %rbp`, or simply `leave` — belongs immediately before the `ret`.

scanf

```
        .data
int_fmt:
        .string "%d" ; pour scanf, c'est un entier
        .text
        .globl main
main: ; main est une fonction, on doit ret
        pushq   $0 ; on alloue un espace sur la stack
        leaq    (%rsp), %rsi ; on met l'adresse dans %rsi
        leaq    int_fmt(%rip), %rdi ; format dans %rdi
        mov     $0, %rax ; on met 0 dans rax
        call    scanf
        popq    %rax
        ret
        .section .note.GNU-stack
```

Line by line, against the convention just stated:

- `pushq $0` allocates eight zeroed bytes on the pile. It does double duty: it is the storage `scanf` will write into, *and* it moves `%rsp` from $\equiv 8$ back to $\equiv 0 \pmod{16}$, so the coming `call` is aligned.
- `leaq (%rsp), %rsi` puts the *address* of that slot into the second registre d'argument. `scanf` wants somewhere to store, so it takes a pointeur — this is why the C call is `scanf("%d", &n)`.
- `leaq int_fmt(%rip), %rdi` puts the address of the format, a chaîne de caractères, into the first registre d'argument.
- `mov $0, %rax` declares zero vector registres used, as every variadic call must.
- `popq %rax` frees the slot — and, as a bonus, leaves the value that was read in `%rax`.

Remark (`%rip`-relative addressing). `leaq int_fmt(%rip), %rdi` computes the address of the label as an offset from the current pointeur d'instruction, so the instruction encodes a displacement rather than an absolute address. This is what makes the code position-independent, and `gcc` produces position-independent exécutables by default; a bare `mov $int_fmt, %rdi` would embed an absolute address and fail to link or relocate. Every reference to a label in `.data` in these examples is written this way.

`printf` avec entier

```
        .data
int_fmt:
        .string "%d\n" ; entier avec retour a la ligne
        .text
        .globl main
main:
        push %rbp
        mov     $42, %rsi ; on met l'entier dans %rsi
        leaq    int_fmt(%rip), %rdi ; format dans %rdi
        mov     $0, %rax ; on met 0 dans rax
        call    printf
        xor     %rax, %rax
        pop %rbp
        ret
        .section .note.GNU-stack
```

The symmetry with `scanf` is the point to remember: `printf` is given the *value* 42 in `%rsi`, whereas `scanf` was given an *address*. The opening `push %rbp` here allocates nothing; it exists to realign `%rsp` on a multiple of sixteen before `call printf`, and the matching `pop %rbp` undoes it. The `xor %rax, %rax` after the call sets the valeur de retour of `main` to zero, the conventional "success" exit status.

`printf` avec string

```
        .data
int_fmt:
        .string "%s\n" ; string avec retour a la ligne
hello_world:
        .string "Hello World!" ; string a afficher
        .text
        .globl main
main:
        push %rbp
        leaq    hello_world(%rip), %rsi ; chaine -> %rsi
        leaq    int_fmt(%rip), %rdi ; format -> %rdi
        mov     $0, %rax ; 0 -> rax
        call    printf
        xor     %rax, %rax
        pop %rbp
        ret
        .section .note.GNU-stack
```

Exactly one *instruction* changes with respect to the integer version: `%rsi` receives an address rather than an immediate, because a chaîne de caractères in C *is* a pointeur. The rest of the difference is data — the format directive changes from `%d` to `%s` to match, and two new lines in `.data` declare the label `hello_world` and the literal it names.

Remark. The label is still called `int_fmt` although it now holds `"%s\n"`. A label is only a name for an address; the assembler attaches no type to it and does not care.

Remark. `.string` emits the bytes of the literal followed by a terminating `NUL`, which is what makes it usable as a chaîne de caractères in C. `.ascii` would emit the bytes alone.

4.5 Quelques exercices

Seven exercises, in increasing order of difficulty:

1. lire un entier n (avec `scanf`) puis l'afficher (avec `printf`) ;
2. lire un entier n puis afficher $2n$;
3. lire un entier n puis afficher $n/2$ s'il est pair et $3n+1$ sinon ;
4. lire un entier n puis afficher $n!$ en utilisant une boucle ;
5. lire un entier n puis afficher $n!$ en utilisant une fonction ;
6. lire trois entiers a, b, c puis afficher $a^b \bmod c$ en utilisant une exponentiation rapide ;
7. lire un entier n puis afficher le triangle de Sierpiński de taille 2^n en utilisant de l'ASCII art.

The last one is recursive in its shape: the triangle T_{N+1} is T_N placed above two side-by-side copies of T_N.

```
N=0        N=1        N=2        N+1
#          #          #          T_N
           ##         ##         T_N T_N
                      # #
                      ####
```

Example (Exercise 1, assembled from the two schemas)**.** Reading an integer and printing it back is the concatenation of the `scanf` skeleton and the `printf` skeleton, sharing the single slot on the pile:

```
        .data
in_fmt:
        .string "%d"
out_fmt:
        .string "%d\n"
        .text
        .globl main
main:
        pushq   $0                  # the slot for n, and %rsp back to 0 mod 16
        leaq    (%rsp), %rsi        # &n            -> 2nd argument
        leaq    in_fmt(%rip), %rdi  # "%d"          -> 1st argument
        mov     $0, %rax            # no %xmm argument
        call    scanf
        movl    (%rsp), %esi        # n             -> 2nd argument
        leaq    out_fmt(%rip), %rdi # "%d\n"       -> 1st argument
        mov     $0, %rax
        call    printf
        popq    %rax                # free the slot
        xor     %rax, %rax          # exit status 0
        ret
        .section .note.GNU-stack
```

Both calls execute with `%rsp` $\equiv 0 \pmod{16}$: the entry offset of eight is cancelled by the single `pushq`, and nothing between the two calls moves `%rsp`.

Remark. The transfer is `movl` into `%esi`, not `movq` into `%rsi`, because the conversion `%d` denotes a 32-bit C `int`: `scanf` writes four bytes and `printf` reads four bytes. Writing a 32-bit registre zeroes the upper half of its 64-bit counterpart, so nothing is left dangling in `%rsi`.

Remark. Exercises 4 and 5 are deliberately the same computation twice. The point of the second is that turning the loop into a function forces every item of this chapter into play at once: the argument arrives in `%rdi`, the result must leave in `%rax`, any accumulator kept across the recursive call must live in a registre sauvegardé par l'appelé or on the pile, and the pile must be aligned at each `call`.

4.6 Faut-il programmer en assembleur ?

Conclusion. Dans la plupart des cas, **non** :

- compliqué à lire ;
- compliqué à écrire ;
- pas portable ;
- du code plus rapide, mais les compilateurs ne sont pas mauvais et il vaut mieux travailler sur les algorithmes.

Ici, **oui** :

- comprendre le CPU ;
- comprendre les points de ralentissement ;
- nécessaire pour le projet.

The honest reading of the first list is that hand-written assembly loses on every axis except one, and loses on that one too as soon as the algorithm is in question: a better asymptotic beats a better inner loop. The second list is why the chapter exists anyway. The calling convention, the frame and the alignement rule are not facts about writing assembly by hand — they are facts about what a compilateur must emit, and the back end built later in this module has to emit them correctly.

5 Grammaires et analyse syntaxique

The lexeur turns a stream of characters into a stream of lexèmes; the *parseur* turns that stream of lexèmes into a tree. This chapter is about the second step, and about the objects that make it possible: grammaires formelles, the restriction to grammaires non contextuelles that makes the problème de l'appartenance decidable at all, ambiguity and how a grammar is rewritten to remove it, a cubic-time recognition algorithm together with a full hand-run trace of it, and finally the linear-time restriction — LL(1) — that real compilateurs actually use. The chapter is written top-down in the sense that it starts with the most general object and removes power at every step, because every restriction buys something: decidability, then polynomial time, then linear time.

5.1 Grammaires formelles

Definition (Grammaire formelle)**.** Une **grammaire formelle** (*formal grammar*) c'est :

- un ensemble de **symboles terminaux** (*terminal symbols*) souvent noté Σ ;
- un ensemble de **symboles non-terminaux** (*non-terminal symbols*), noté N ;
- un **symbole de départ** (*start symbol*) S ;
- un ensemble de **règles de production** (*production rules*) $(\Sigma \cup N)^+ \to (\Sigma \cup N)^*$.

Notation. On donne souvent les symboles Σ en minuscules et ceux de N en majuscules. La grammaire est donc *juste un ensemble de règles* — Σ, N and S are read off the rules themselves.

The left-hand side of a rule is a non-empty *mot* of symbols, not a single non-terminal. That is the whole generality of the definition, and it is exactly what will be given up in the next subsection.

Example (A grammar that is not context-free)**.** $S \to aSb, \quad S \to \epsilon, \quad ab \to ba.$

Definition (Langage d'une grammaire)**.** Le **langage** $L(G)$ d'une grammaire G c'est l'ensemble des mots de Σ^* que l'on peut obtenir en appliquant des **règles de réécriture** (*rewriting rules*) depuis la chaîne contenant S (and nothing else).

Example (***aababb*** belongs to the language above)**.** *aababb* est dans le langage de $S \to aSb$, $ab \to ba$ car

$$S \to aSb \to aaSbb \to aaaSbbb \to aaabbb \to aababb.$$

The first three steps use $S \to aSb$, the fourth uses $S \to \epsilon$, and the last one uses $ab \to ba$ on the middle of *aaabbb*.

Remark. Quel est le langage des mots reconnus par cette grammaire ? The answer is the set of words over $\{a, b\}$ containing as many a as b: the first two rules generate exactly $a^n b^n$, and $ab \to ba$ lets a b migrate leftwards past an a as often as one likes, which reaches every arrangement of n letters a and n letters b (run the rule backwards and one is merely bubble-sorting the word into $a^n b^n$). The rule $ab \to ba$ can never fire before the S is gone, since no mot containing S has an a immediately followed by a b.

Le problème de l'appartenance

Example (Problème de l'appartenance (*membership problem*))**.** Étant donné w et G, décider si $w \in L(G)$.

Theorem. *For general grammaires formelles the problème de l'appartenance is* ***indécidable*** *(undecidable).*

$\Rightarrow$ on va devoir restreindre l'ensemble des grammaires. Il existe plusieurs restrictions utiles et étudiées. The rest of the chapter is a tour of the two that matter for a compilateur.

5.2 Grammaires non contextuelles

Definition (Grammaire non contextuelle)**.** Une **grammaire non contextuelle** (*context-free grammar*, CFG) ou *grammaire hors-contexte* ou *grammaire algébrique* c'est une grammaire dans laquelle les règles de production sont toutes de la forme

$$N \to (\Sigma \cup N)^*.$$

The name says what has been given up: the left-hand side is a single non-terminal, so a rule may be applied to an occurrence of N *without looking at its context.* The grammar $ab \to ba$ of the previous subsection is excluded.

Example. $S \to S + S, \quad S \to S * S, \quad S \to 1.$

For this class the problème de l'appartenance is **décidable et polynomial** — the algorithm and its exponent are the subject of the next two subsections.

Arbre de dérivation

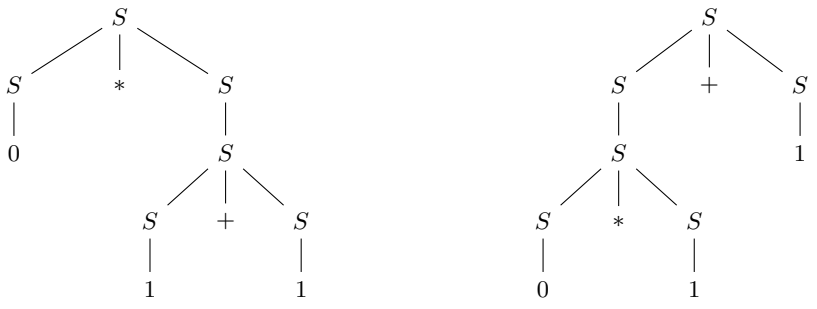
Definition (Arbre de dérivation)**.** Pour G une CFG, un **arbre de dérivation** (*derivation tree*), c'est un arbre A dans lequel :

- les feuilles sont étiquetées par des terminaux ;
- les nœuds internes sont étiquetés par des non-terminaux ;
- la racine est étiquetée par S ;
- les enfants $n_1 \dots n_k$ d'un nœud étiqueté par X doivent être étiquetés par $s_1 \dots s_k$ tel que $X \to s_1 \dots s_k \in G$.

Le **mot de *A*** c'est le mot des étiquettes des feuilles dans l'ordre.

Remark. $w \in L(G)$ si et seulement si w est le mot d'un arbre de dérivation. On dit qu'une grammaire est **non-ambiguë** (*unambiguous*) s'il existe un *unique* arbre de dérivation pour chaque mot.

The arbre de dérivation, not the derivation, is the object a compilateur wants: the order in which the rules were applied is irrelevant, the shape of the tree is the meaning. A word with two arbres de dérivation is a program with two meanings.



Both trees have word $0 * 1 + 1$, so the grammar $S \to S + S \mid S * S \mid 1 \mid 0$ is ambiguous, and it is ambiguous in the worst possible way: the two trees are the two different arithmetic values.

Remark. The same grammar has smaller trees, and their words are read off the leaves in the same way. The tree $S \to S + S$ whose left child derives 1 and whose right child derives 0 has a 0 as its right leaf, so its word is $1 + 0$. The two trees above both have word $0 * 1 + 1$, as their leaves show.

5.3 Grammaire ambiguë

On s'attend en général à ce que la structure des programmes écrits ne soit pas ambiguë. The canonical counter-example is the dangling `else`:

```
if(cond1)
  if (cond2)
    printf("a");
else
    print("b");
```

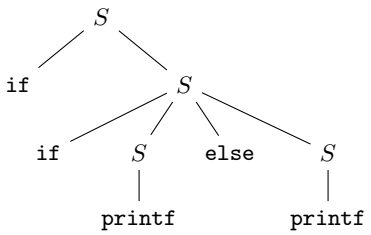
`cond1`	`cond2`	affichage
Vrai	Vrai	"a"
Vrai	Faux	?
Faux	...	?

The indentation suggests the `else` belongs to the outer `if`; the language says otherwise. **La syntaxe du C est claire, le `else` se raccroche au `if` le plus imbriqué.** The code therefore parses as `if(cond1) { if(cond2) printf("a"); else print("b"); }`, so the second row prints `"b"` — the inner `else` fires — and the third row prints nothing at all, the whole outer statement being skipped. These are the answers the rule forces in the two `?` cells of the table. That the rule has to be stated at all is a decision the language designer had to take, and it has to be forced by the grammar rather than by a footnote.

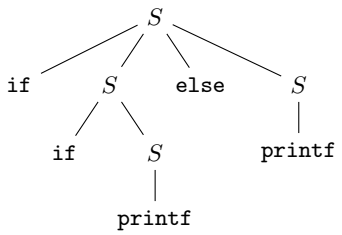
Modéliser le `else` ambigu

Modélisons l'exemple `if if printf else printf`. Grammaire :

$$S \to \texttt{printf}, \qquad S \to \texttt{if}\, S, \qquad S \to \texttt{if}\, S\, \texttt{else}\, S.$$



`else` on the inner `if`
(the C reading)



`else` on the outer `if`
(the indentation's reading)

Example (Exercice — rendre la grammaire non ambiguë)**.** Modifier la grammaire pour la rendre non ambiguë.

Example (Solution)**.**

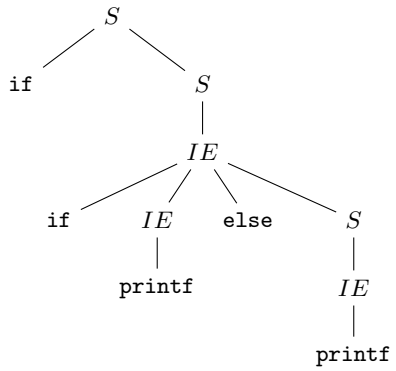
$$S \to \texttt{if}\, S \qquad IE \to \texttt{printf}$$
$$S \to IE \qquad IE \to \texttt{if}\, IE\ \texttt{else}\, S$$

A new non-terminal IE is introduced and the then-branch of an `if`…`else` is restricted to it: only IE, never S, may sit between an `if` and its `else`. That is what puts the `else` on the inner `if` in the tree below, and it disambiguates the example the exercise was set on.

Remark. It does *not* disambiguate the grammar in general, and it is worth seeing why, because the tempting justification — that IE is *closed*, i.e. cannot swallow a following `else` — is false. The rule $IE \to \texttt{if}\, IE\ \texttt{else}\, S$ ends in S, and $S \to \texttt{if}\, S$, so an IE can itself end in an unmatched `if`. The shortest witness is

```
if if printf else if printf else printf
```

which still has **two** arbres de dérivation: the final `else` can bind the third `if` (via $S \to \texttt{if}\, S$ with the inner S an IE), or bind the outermost one (via $S \to IE$ with `if if printf else if printf` derived as the inner IE), leaving the third `if` dangling. Every shorter mot has a unique tree, which is why a single illustrative derivation, like the one drawn below, looks conclusive. Closing the construction properly requires the else-branch to be closed too: $IE \to \texttt{if}\, IE\ \texttt{else}\, IE$.



Gérer la précédence des opérateurs

The same disease, in its arithmetic form. Take

$$S \to \texttt{int}, \qquad S \to S + S, \qquad S \to S * S, \qquad S \to S \hat{} S,$$

and again the exercise is: modifier la grammaire pour la rendre non ambiguë.

Example (A first solution, and why it is wrong)**.**

$$S \to E + S, \qquad S \to E * S, \qquad S \to E\hat{}S, \qquad E \to \texttt{int}.$$

Forcing the left operand down to a single `int` does remove the ambiguity — every operator now cuts the expression at its first occurrence — but it removes it the wrong way. $6 + 5 * 4\hat{}3 * 2 + 1$ est lu comme

$$6 + \big(5 * (4\hat{}(3 * (2 + 1)))\big) \qquad \text{whereas one would rather want} \qquad 6 + \big(5 * (4\hat{}3) * 2\big) + 1.$$

All three operators have become right-associative and all three have the same precedence, because a single non-terminal S is doing the work of three different levels.

Example (A better solution: stratify by precedence)**.** Attribuer une **précédence** à chaque opérateur, 0 pour $+$, 1 pour $*$ et 2 pour $\hat{}$. Then give each level its own non-terminal S_i:

$$\begin{array}{lll} S_0 \to S_0 + S_1 & S_1 \to S_1 * S_2 & S_2 \to S_3\hat{}S_2 \\ S_0 \to S_1 & S_1 \to S_2 & S_2 \to S_3 \\ & & S_3 \to \texttt{int} \end{array}$$

A level-i expression is a chain of level-i operators whose operands are level-$(i{+}1)$ expressions, and the "escape" rule $S_i \to S_{i+1}$ lets a tighter-binding expression be used wherever a looser one is expected. The precedence is now structural: $*$ cannot appear at the top of an S_0 chain except inside an S_1.

Remark (Associativité comes for free). Cela gère même l'**associativité** (*associativity*) droite ou gauche — the knob is which side the recursion is on. With

$$\begin{array}{lll} S_0 \to S_0 + S_1 & S_1 \to S_1 * S_2 & S_2 \to S_3\hat{}S_2 \\ S_0 \to S_0 - S_1 & S_1 \to S_1 / S_2 & S_2 \to S_3 \\ S_0 \to S_1 & S_1 \to S_2 & S_3 \to \texttt{int} \end{array}$$

one gets

$$2\hat{}3\hat{}4 \;\Rightarrow\; 2^{(3^4)} \neq (2^3)^4, \qquad 1 - 2 - 3 \;\Rightarrow\; (1 - 2) - 3.$$

S_0 recurses on the *left*, so $+$ and $-$ are left-associative; S_2 recurses on the *right*, so $\hat{}$ is right-associative. That is the arithmetic one wants, and it costs one character of grammar.

5.4 Reconnaître une grammaire non contextuelle

Binarisation

Definition (CFG binaire)**.** Une CFG est **binaire** si toutes les règles de production sont de la forme

$$N \to \epsilon \cup (N \cup \Sigma) \cup (N \cup \Sigma)^2,$$

i.e. if every right-hand side is empty, a single symbol, or a pair of symbols.

Theorem (Binarisation)**.** *Toute CFG G admet une CFG binaire G' telle que $L(G) = L(G')$.*

Proof Sketch. On répète la procédure suivante : si G contient $X \to s_1 \dots s_k$ (with $k > 2$), on réécrit cette règle en les deux règles $X \to s_1 F$ et $F \to s_2 \dots s_k$ avec F un symbole non-terminal frais. Each step decreases the total length of the right-hand sides that are too long, so the procedure terminates. ∎

Remark. This is *not* Chomsky normal form: ϵ-productions and unit productions $N \to M$ survive binarisation, and the recognition algorithm below is written to cope with both. The price of binarisation is that the arbres de dérivation of G' are not those of G — the fresh symbols F appear as spurious internal nodes, which have to be flattened out again if the tree is what one is after.

Algorithme de reconnaissance

Definition (La relation *match*)**.** Étant donné un mot $w = w_0 \dots w_{n-1}$ et une grammaire binaire G, on dit que $(\alpha, i, j) \in match$ pour α un symbole de G et $0 \leq i \leq j \leq n$ si α peut se dériver en $w_i \dots w_{j-1}$.

Notation. The triple is written both as (α, i, j) and as (i, j, α); they are the same object. The convention throughout is that the span is *half-open*: (i, j) covers $w_i \dots w_{j-1}$, so a single lexème w_i has span $(i, i+1)$ and the whole word has span $(0, n)$.

Definition (*match* comme plus petit point fixe (*fixpoint*))**.** L'ensemble *match* est le plus petit ensemble qui contient :

- (i, i, α) si $\alpha \to \epsilon \in G$;
- $(i, i+1, w_i)$ pour tout $i < n$;
- (i, j, α) si $\alpha \to \beta \in G$ et $(i, j, \beta) \in match$;
- (i, j, α) si $\alpha \to \beta\gamma \in G$, $(i, k, \beta) \in match$ et $(k, j, \gamma) \in match$.

Theorem. *On a $w \in L(G)$ si et seulement si $(S, 0, n) \in match$.*

Calcul de *match*

The definition is a point fixe; the algorithm is the obvious worklist that saturates it. Initialiser *match* avec

- (i, i, α) si $\alpha \to \epsilon \in G$;
- $(i, i+1, w_i)$ pour tout $i < n$.

Ensuite on traite les éléments (i, j, α) de *match un par un* en appliquant les règles suivantes :

- si $\beta \to \alpha \in G$ on ajoute (i, j, β) ;
- si $\beta \to \alpha\gamma \in G$, pour $k \geq j$ t.q. $(j, k, \gamma) \in match$ on ajoute (i, k, β) ;
- si $\beta \to \gamma\alpha \in G$, pour $k \leq i$ t.q. $(k, i, \gamma) \in match$ on ajoute (k, j, β).

Pour (i, j) donnés, chaque production est considérée au plus $2n$ fois. Hence

$$\boxed{\text{Complexity } O(n^3 \times |G|)}.$$

Remark (Which algorithm this is). This is the **CYK** algorithm (Cocke–Younger–Kasami), in agenda form. The textbook presentation fills a triangular table indexed by (i, j) in order of increasing span length and demands Chomsky normal form; the presentation here computes exactly the same relation *match* as a least point fixe driven by a worklist, and asks only for binarisation. The two shapes of the last rule — extend to the right with $\beta \to \alpha\gamma$, extend to the left with $\beta \to \gamma\alpha$ — are what replaces the ordered sweep: since items are not produced in order of length, a new item must be offered both as a left part and as a right part of every binary production. The factor $2n$ in the count is precisely those two directions times the n possible values of k.

Retrouver l'arbre de dérivation

Recognition is not analyse syntaxique: $(S, 0, n) \in match$ answers yes or no and returns no tree. One recovers the tree by carrying a witness alongside every item. On ajoute à *match* un **tableau** D avec

- $D[i, i, \alpha] = \alpha$ si $\alpha \to \epsilon \in G$;
- $D[i, i+1, w_i] = w_i$ pour tout $i < n$.

Ensuite pour les éléments $(i, j, \alpha) \to t$ de *match*, on a :

- si $\beta \to \alpha \in G$ alors $D[i, j, \beta] = \beta(t)$;
- si $\beta \to \alpha\gamma \in G$, pour $k \geq j$ t.q. $D[j, k, \gamma] = t' \neq \varnothing$ alors $D[i, k, \beta] = \beta(t, t')$;
- si $\beta \to \gamma\alpha \in G$, pour $k \leq i$ t.q. $D[k, i, \gamma] = t' \neq \varnothing$ alors $D[k, j, \beta] = \beta(t', t)$.

Pour (i, j) donnés, chaque production est considérée au plus $2n$ fois. The complexity is therefore unchanged: $O(n^3 \times |G|)$.

Remark. In the unit bullet, $D[i, j, \beta] = \beta(t)$ is a β-node with the α-subtree t as its only child. The node is labelled β because the item being filled is the one for β, exactly as in the two binary bullets; α labels only the root of its child.

Remark. D stores *one* tree per cell, so it returns one parse of an ambiguous word and silently drops the others. Storing the set of all trees would cost exponential space; what is stored instead, in a real implementation, is the set of backward *pointeurs* per cell — the shared packed parse forest — which stays cubic.

5.5 Faisons tourner l'algorithme

Here the algorithm is run by hand on the grammar below, in its **non-binarised** version: the right-hand sides are matched whole against consecutive spans rather than two symbols at a time. Nothing else changes — the items are still triples (i, j, α) and the closure rules are still the same — but the trace stays readable, because the rule $S \to$ `let var` $= S$ `in` S is applied in one step instead of five.

$$
\begin{array}{lll}
S \to \texttt{let var} = S \texttt{ in } S & P \to T & T \to \texttt{var} \\
S \to P & P \to T * P & T \to \texttt{int} \\
S \to P + S & &
\end{array}
$$

The input is `1+let x = 2 in x*3+4`, which the lexeur has already turned into twelve lexèmes; $n = 12$ and the boundaries run from 0 to 12.

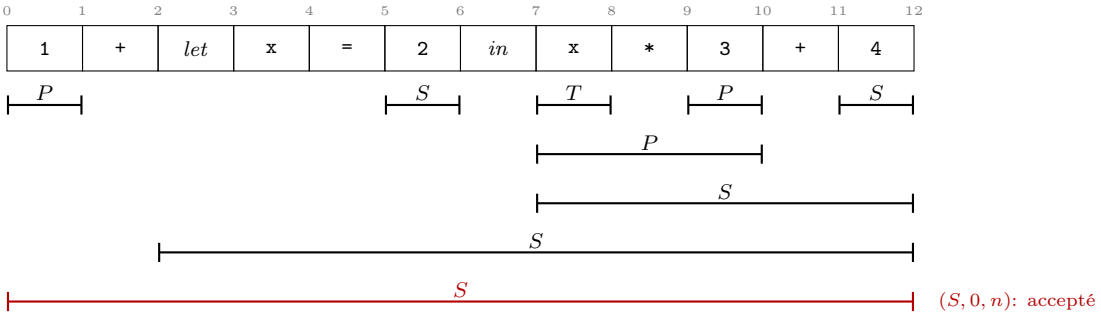
`1`	`+`	*let*	`x`	`=`	`2`	*in*	`x`	`*`	`3`	`+`	`4`
0	1	2	3	4	5	6	7	8	9	10	11

The trace works from the right end of the input towards the left. At each step the items that can no longer be useful are dropped from the display, which is why the list does not simply grow.

	Item derived	Justification
1	$(11, 12, T)$, $(11, 12, P)$, $(11, 12, S)$	`4` $\leftarrow T \leftarrow P \leftarrow S$
2	$(9, 10, T)$, $(9, 10, P)$	`3` $\leftarrow T \leftarrow P$
3	$(7, 8, T)$	`x` $\leftarrow T$ (rule $T \to$ `var`)
4	$(7, 10, P)$	$P \to T * P$ on $(7, 8, T)$, $(8, 9, *)$, $(9, 10, P)$
5	$(7, 12, S)$	$S \to P + S$ on $(7, 10, P)$, $(10, 11, +)$, $(11, 12, S)$
6	$(5, 6, S)$	`2` $\leftarrow T \leftarrow P \leftarrow S$
7	$(2, 12, S)$	$S \to$ `let var` $= S$ `in` S on $(5, 6, S)$ and $(7, 12, S)$
8	$(0, 1, P)$	`1` $\leftarrow T \leftarrow P$
9	$(0, 12, S)$	$S \to P + S$ on $(0, 1, P)$, $(1, 2, +)$, $(2, 12, S)$

$(S, 0, 12) \in$ *match* with $n = 12$, so the word is accepté.

Remark (Checking the trace). Two checks apply to any trace. First, a span always has $i \leq j$: the lexème `3` sits at index 9, so step 2 yields $(9, 10, T)$ alongside $(9, 10, P)$. Second, every justification names productions of the grammar: the last step is $S \to P + S$ applied to $(0, 1, P)$, $(1, 2, +)$ and $(2, 12, S)$.



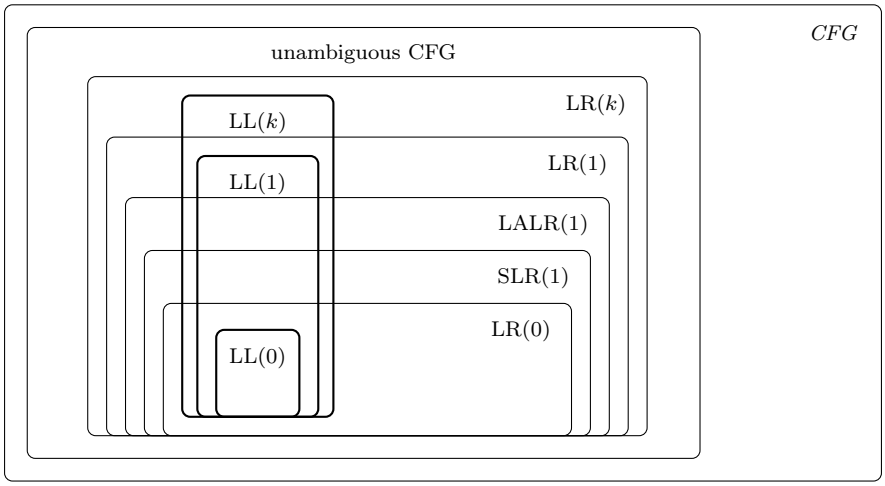
Remark (Why is this grammar not ambiguous?). The answer is that the three non-terminaux are *stratified* exactly as in the precedence construction above: T derives a single lexème, P derives a $*$-chain of Ts, and S derives a $+$-chain of Ps or a `let`-expression. So for any given span at most one production can apply: a span containing a top-level $+$ is not a P, and no P or T can start with `let`. Where a rule mentions the same non-terminal twice — $T * P$, $P + S$ — the recursion is on the right only, so the cut is forced at the *first* operator of the chain and there is nothing left to choose. Both operators therefore come out right-associative.

5.6 Utilisation en pratique

La reconnaissance de mots par CFG est en $O(n^3 \times |G|)$, cela peut sembler raisonnable mais :

- les fichiers de code contiennent parfois plusieurs milliers de lignes (et plus si on compte les entêtes inclus) ;
- cela peut faire de l'ordre de la centaine de milliers voire du million de symboles ;
- un processeur des années 70 faisait $\sim 10^6$ opérations par seconde $\Rightarrow$ il aurait fallu 30 000 ans pour faire 10^{18} opérations ;
- un processeur moderne fait $\sim 10^9$ opérations par seconde $\Rightarrow$ il faut 32 ans pour faire 10^{18} opérations.

Remark. 10^{18} is n^3 for $n = 10^6$, so the two lines are the same computation on the same file. The point is the exponent, not the constant: four decades of hardware progress moved a cubic algorithm from "geologically impossible" to "merely impossible". Hence les langages utilisent souvent des grammaires avec **test linéaire** (*linear test*).



Read the picture as inclusions of *grammar* classes, not of languages: $\mathrm{LR}(0) \subset \mathrm{SLR}(1) \subset \mathrm{LALR}(1) \subset \mathrm{LR}(1) \subset \mathrm{LR}(k)$, all of them non-ambiguës, and the LL boxes cut across them — $\mathrm{LL}(0) \subset \mathrm{LL}(1) \subset \mathrm{LL}(k)$ with each

LL(i) inside the corresponding LR(i). The *lookahead* k is the number of lexèmes the parser is allowed to peek at, and the classes with $k = 1$ are the ones for which parser generators exist.

5.7 Analyse syntaxique descendante

Comment reconnaître *rapidement* une telle grammaire ? Take the stratified expression grammar, reduced to two operators:

$$S_0 \to S_1 + S_0 \qquad S_1 \to S_2 * S_1 \qquad S_2 \to \texttt{int}$$
$$S_0 \to S_1 \qquad S_1 \to S_2$$

Example (Recursive descent by hand)**.** With two primitives on the lexème stream — `peek()` donne le prochain symbole à lire sans le consommer, `eat()` donne et consomme le prochain symbole — one function per non-terminal suffices.

```
peek() # donne le prochain symbole a lire sans le consommer
eat()  # donne et consomme le prochain symbole

def s0():                def s1():                def s2():
  a = s1()                 a = s2()                 a = peek()
  if peek() != '+':        if peek() != '*':        assert(a == 'int')
    return a                 return a               eat()
  eat()                    eat()                    return a
  return (a,'+',s0())      return (a,'*',s1())
```

Each function returns the subtree it has just recognised; the tuple `(a,'+',s0())` *is* the node of the arbre de dérivation. Note that the recursive call sits in the returned tuple, i.e. at the right of the node, matching the right recursion of $S_0 \to S_1 + S_0$.

Remark. This is why the method is called **descendante** (*top-down*): the parser starts at S and descends, choosing a production before it has seen what it derives. The CYK algorithm of the previous section is the opposite — bottom-up, combining items already recognised. Analyse syntaxique descendante is linear and fits in twelve lines; the price is that the choice of production has to be decidable from the next lexème alone, which is exactly the LL(1) condition.

Pourquoi cela marche et peut-on généraliser ?

Commençons par réécrire la grammaire, so that the choice is never between two productions that begin the same way — **factorisation à gauche** (*left factoring*):

$$S_0 \to S_1 F_0 \qquad S_1 \to S_2 F_1 \qquad S_2 \to \texttt{int}$$
$$F_0 \to +S_0 \qquad F_1 \to *S_1$$
$$F_0 \to \epsilon \qquad F_1 \to \epsilon$$

The common prefix S_1 of the two S_0-rules has been pulled out, and what was the choice between $S_1 + S_0$ and S_1 has become the choice, inside F_0, between consuming a + and producing nothing. The rewritten grammar is a transcription of the code: the `if peek() != '+': return a` is exactly $F_0 \to \epsilon$.

Definition (Les trois ensembles)**.**

- *Null*: the set of non-terminaux X such that $X \Rightarrow \epsilon$;
- *Suivant*(X) (*follow*) : les terminaux qui peuvent suivre X ;
- *Premier*(X) (*first*) : les terminaux qui peuvent commencer un mot dérivé de X.

Remark. Null is a set of *non-terminaux*. A terminal derives only itself, so no *terminal* X satisfies $X \Rightarrow \epsilon$, and a *Null* made of terminaux would always be empty. The computed value $\{F_0, F_1\}$ below is indeed a set of non-terminaux.

Example (The three sets on the factored grammar)**.**

$$Null = \{F_0, F_1\}$$

$$\begin{aligned} Premier(S_i) &= \{\texttt{int}\} & Suivant(S_0) &= Suivant(F_0) = \{\} \\ Premier(F_0) &= \{+\} & Suivant(S_1) &= Suivant(F_1) = \{+\} \\ Premier(F_1) &= \{*\} & Suivant(S_2) &= \{*, +\} \end{aligned}$$

Every S_i begins with `int` because every chain bottoms out at $S_2 \to \texttt{int}$. $Suivant(S_0)$ is empty because S_0 is the symbole de départ and nothing follows the whole input. $Suivant(S_2) = \{*, +\}$ collects $Premier(F_1) = \{*\}$ from $S_1 \to S_2 F_1$ and, because F_1 is nullable, $Suivant(S_1) = \{+\}$ as well.

Definition (Grammaire LL(1))**. Grammaire LL(1)** : grammaire qui ne contient pas $X \to \alpha$ et $X \to \beta$ ($\alpha \neq \beta$) telles que (any one of the following is enough) :

- α et β dérivent toutes deux ϵ ;
- $Premier(\alpha) \cap Premier(\beta) \neq \varnothing$;
- α dérive ϵ et $Premier(\beta) \cap Suivant(X) \neq \varnothing$.

$\Rightarrow$ définition alternative de LL(1) : **le prochain symbole détermine la production à utiliser**. Each of the three clauses is one way for two productions of the same X to be indistinguishable on one lexème of lookahead — both empty, both able to start with the same terminal, or one empty and the other able to start with something that could equally well have been the continuation after X.

The factored grammar above violates none of the three, so it is **LL(1)** — and that is the theorem behind the twelve lines of code: for an LL(1) grammar the recursive-descent parser is well defined, deterministic, and runs in time linear in the number of lexèmes.

Exercice — un analyseur syntaxique descendant pour la grammaire `let`

Example. Comprendre pourquoi cette grammaire n'est pas LL(1) puis la modifier pour la rendre LL(1) et enfin écrire un analyseur syntaxique descendant.

$$\begin{array}{lll} S \to \texttt{let var} = S \texttt{ in } S & P \to T & T \to \texttt{var} \\ S \to P & P \to T * P & T \to \texttt{int} \\ S \to P + S & & \end{array}$$

Terminaux : `var`, `int`, `+`, `*`, `let`, `=`, `in`.

Example (Worked answer)**.** What follows applies the method of the previous subsection.

Why it fails. $Premier(P) = Premier(T) = \{\texttt{var}, \texttt{int}\}$, so the two productions $S \to P$ and $S \to P + S$ have intersecting *Premier* sets and the second clause of the definition fires: on seeing `var` the parser cannot tell which to choose. The pair $P \to T$ and $P \to T * P$ fails for the same reason. The `let` production is fine — it is the only one beginning with `let`.

The fix. Left-factor both offending pairs, exactly as S_0 was factored into $S_0 \to S_1 F_0$:

$$\begin{array}{lll} S \to \texttt{let var} = S \texttt{ in } S & P \to TG & T \to \texttt{var} \\ S \to PF & G \to *P & T \to \texttt{int} \\ F \to +S & G \to \epsilon & \\ F \to \epsilon & & \end{array}$$

Now $Premier(\texttt{let} \dots) = \{\texttt{let}\}$ is disjoint from $Premier(PF) = \{\texttt{var}, \texttt{int}\}$; F and G each offer a production starting with a distinct operator against an ϵ-production, and $+ \notin Suivant(F)$, $* \notin Suivant(G)$, so the third clause does not fire either.

The parser.

```
def s():
  if peek() == 'let':
    eat()                       # let
    v = eat()                   # var
    eat()                       # =
    e1 = s()
    eat()                       # in
    return ('let', v, e1, s())
  a = p()
  if peek() != '+':             # F -> epsilon
    return a
  eat()
  return (a,'+',s())            # F -> + S

def p():
  a = t()
  if peek() != '*':             # G -> epsilon
    return a
  eat()
  return (a,'*',p())            # G -> * P

def t():
  a = peek()
  assert(a == 'var' or a == 'int')
  eat()
  return a
```

One function per non-terminal of the *unfactored* grammar; the factored F and G appear as the trailing `if` inside `s()` and `p()` rather than as functions of their own, which is the usual way the factorisation is written back into code.

5.8 De nombreuses classes...

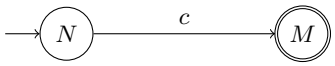
Il existe d'autres classes intéressantes, considérons maintenant les CFG où toutes les productions sont de la forme

$$N \to \Sigma \times N \cup \epsilon,$$

that is, every right-hand side is either a terminal followed by a single non-terminal, or empty.

Theorem. *These are exactly the* ***langages réguliers*** *(regular languages).*

Remark. Such grammars are the **right-linear** ones, and the correspondence with automates finis is immediate: read each non-terminal as an état, each rule $N \to cM$ as a transition $N \xrightarrow{c} M$, and each rule $N \to \epsilon$ as "N is accepting". The symbole de départ is the état initial. Nothing is lost and nothing is added, which is why the next chapter can speak of automates and of grammars interchangeably.



The rules $N \to cM$ and $M \to \epsilon$, read as an automate.

Conclusion. Four thresholds structure the chapter, and each one is a restriction paid for with an algorithm. General grammaires formelles: problème de l'appartenance indécidable. Restrict the left-hand side to a single non-terminal and one gets the grammaires non contextuelles, where the problème de l'appartenance is decided in $O(n^3 \times |G|)$ by saturating the relation *match* — CYK — after binarising the productions. Cubic is still hopeless on a file of a million lexèmes, so restrict further to the grammars whose next production is determined by the next lexème: LL(1), tested with *Null*, *Premier* and *Suivant*, parsed in linear time by one recursive function per non-terminal. Restrict once more, to right-linear productions, and one has fallen out of analyse syntaxique altogether into the langages réguliers and automates finis — which is where the lexeur lives, and where the next chapter begins.

The recurring technique, across all four levels, is *rewriting the grammar rather than the parser*: stratify by precedence to fix associativité, introduce a closed non-terminal to fix the dangling `else`, binarise to fit CYK, left-factor to fit LL(1). The language recognised never changes; only the shape of the trees, and the algorithm that can afford to build them.

6 Langages réguliers et analyse lexicale

The chapter on analyse syntaxique ended by narrowing grammaires non contextuelles until nothing much was left: les CFG où toutes les productions sont de la forme

$$N \;\to\; \Sigma \times N \;\cup\; \epsilon,$$

that is, a terminal followed by a single non-terminal, or nothing at all. That class has a name — *les langages réguliers*. This chapter is that class seen from three sides at once: as expressions régulières, which a human writes; as automates, which a machine runs; and as the front end of the compilateur which uses both, the *lexeur*. It closes on writing a lexeur from scratch.

6.1 Langages réguliers

Definition (Diverses définitions)**.** On peut définir les langages réguliers de diverses manières :

- via les **opérations rationnelles** (*rational operations*) ;
- par les expressions régulières ;
- par les automates ;
- comme image inverse de **monoïdes finis** (*finite monoids*) ;
- par la **logique** (*logic*) ;
- etc.

Remark. The list is not padding. A class of languages with one definition is a curiosity; a class with five unrelated-looking definitions that all pick out the same sets is a robust object, and each definition is good at a different job. The right-linear grammars above are a sixth entry, reached from the side of analyse syntaxique. Only two of them are developed here, and they are chosen for opposite reasons: expressions because they are what one writes, automates because they are what one runs.

6.2 Expressions régulières

Syntaxe

Definition (Expressions régulières, inductivement)**.** On définit inductivement les expressions régulières :

$r ::= \varnothing$	langage vide — the empty language
ϵ	mot vide — the empty word
c	c un caractère — a character
$[c_1 - c_2]$	une classe de caractères — a character class
$r \mid r$	union
$r\,r$	concaténation
$r*$	répétition (zéro ou plus)
$r+$	répétition (au moins une fois)

Eight cases, of which the first four are atoms and the last four are constructors. Note that this is a definition of *syntax* only: at this stage $r*$ is a tree node with one child, nothing more. What it denotes comes next, and is a separate definition.

Remark. $\varnothing$ and ϵ are not the same expression and not the same language. $\varnothing$ denotes the language with no words at all; ϵ denotes the language whose single word is the word of length zero. Concatenating anything with $\varnothing$ gives $\varnothing$; concatenating anything with ϵ changes nothing. The distinction matters as soon as one writes a lexeur rule that can match nothing.

Le langage associé

Definition (Langage associé)**.** On peut aussi définir le **langage associé** (*associated language*) :

Expression r	Langage $L(r)$
$\varnothing$	$\varnothing$
ϵ	$\{\epsilon\}$
c	$\{c\}$
$[c_1 - c_2]$	$\{c \mid c_1 \leq c \leq c_2\}$
$r_a \mid r_b$	$L(r_a) \cup L(r_b)$
$r_a r_b$	$\{w_a w_b \mid w_a \in L(r_a),\ w_b \in L(r_b)\}$
$r*$	$\{w_1 \dots w_k \mid k \geq 0 \wedge \forall i,\ w_i \in L(r)\}$
$r+$	$\{w_1 \dots w_k \mid k \geq 1 \wedge \forall i,\ w_i \in L(r)\}$

The whole table is one structural recursion: the language of a compound expression is built out of the languages of its parts, so L is total and unambiguous on the syntax above.

Remark. The star and the plus differ in exactly one character of the definition, $k \geq 0$ against $k \geq 1$, so $r+$ is redundant: $L(r\, r*) = L(r+)$. It is kept because it is the case one actually writes — an integer literal is $[0-9]+$, never $[0-9][0-9]*$ — and an implementation that omits the constructor simply builds it, writing `Concat(digit, Star(digit))` for `digits`, as in the examples below.

Remark. The character class $[c_1 - c_2]$ is defined by the *order* on characters, $c_1 \leq c \leq c_2$: it is a range in the character encoding, not an arbitrary set. That is why `[a-z]` and `[A-Z]` have to be written as two classes joined by a union — in ASCII they are not contiguous.

Des exemples

The examples are given not as chaînes de caractères in concrete syntax but as constructor calls: an expression is a value of an algebraic data type with one constructor per line of the grammar.

```
digit = CharRange('0','9') # one digit
digits = Concat(digit, Star(digit))
letter = Union(CharRange('a','z'),CharRange('A','Z'))
let_tok = Concat(Char('l'), Concat(Char('e'),Char('t')))
in_tok = Concat(CharRange('i','i'),CharRange('n','n'))
opp = Char('+')
opf = Char('*')
opm = Char('-')
ope = Char('=')
ident = Concat(letter,Star(Union(digit,letter)))
space = Union(Char(' '),Char('\t'))
```

These are the very definitions that the lexeur written at the end of this chapter starts from. Two details of the encoding are worth reading. `let_tok` shows that concatenation is binary, so a three-letter keyword is a nest of two `Concat`s; the surface syntax `let` hides an association choice that the data type must make explicit. And `in_tok` uses `CharRange('i','i')` where `Char('i')` would do — a degenerate class of one character, harmless because $c \leq c \leq c$.

Remark. The constructor set is `Char`, `CharRange`, `Union`, `Concat`, `Star`: five of the eight cases of the grammar. There is no constructor for $\varnothing$, none for ϵ and none for $r+$, so an implementation of the lexeur only has to handle five cases — with $r+$ spelled `Concat(r, Star(r))` as in `digits`.

6.3 Les reconnaître

Knowing what $L(r)$ *is* does not say how to test membership. A recogniser can work directly on the syntax tree, by mutual recursion over a fixed word.

Definition (La fonction match)**.** Étant donné un mot $w_0 \dots w_{n-1}$ fixé on définit

$$\mathrm{match}(R, i) \;=\; \{\, j \mid w_i \dots w_{j-1} \in L(R) \,\}$$

— the set of positions at which a factor of w starting at i and matched by R may end. On a alors :

$$\begin{aligned}
\mathrm{match}(\epsilon, i) &= \{i\} \\
\mathrm{match}(c, i) &= \{i+1 \mid \text{si } w_i = c\} \\
\mathrm{match}([c_1, c_2], i) &= \{i+1 \mid \text{si } w_i \in [c_1, c_2]\} \\
\mathrm{match}(R_a \mid R_b, i) &= \mathrm{match}(R_a, i) \cup \mathrm{match}(R_b, i) \\
\mathrm{match}(R_a / R_b, i) &= \textstyle\bigcup_{j \in \mathrm{match}(R_a, i)} \mathrm{match}(R_b, j) \\
\mathrm{match}(R*, i) &= \text{point fixe}_X \Big(\{i\} \cup_{j \in X} \mathrm{match}(R, j) \Big)
\end{aligned}$$

Notation. The fifth line is the concatenation case: it writes R_a/R_b where the grammar writes R_aR_b, the slash standing in for the invisible concatenation operator. One may also write $\mathit{matches}(R, w, j)$ with the word passed explicitly; since w is fixed throughout, it is the same function as $\mathrm{match}(R, j)$. Note also that the word is indexed here from 0, as $w_0 \dots w_{n-1}$, whereas the automate and the lexeur below index it from 1, as $w_1 \dots w_n$. Neither convention is wrong; only mixing them inside one implementation is.

The shape of these equations is what makes them implementable. Every case returns a *set* of end positions, not a yes/no answer, because an expression régulière can match several prefixes of the same suffix at once, and the appelant — the lexeur — is going to want the largest of them. Union takes the union of the two sets. Concatenation is a relational composition: run R_a from i, and from every place it can stop, run R_b.

Example (Several answers at once)**.** Take $w =$ `let` (so $n = 3$) and $R =$ `ident` $= \mathit{letter}\,(\mathit{digit} \mid \mathit{letter})*$. Then $\mathrm{match}(\mathit{letter}, 0) = \{1\}$, and starring the tail from position 1 reaches 1, 2 and 3, so

$$\mathrm{match}(R, 0) = \{1, 2, 3\},$$

because `l`, `le` and `let` are all identifiers. The recogniser answers "it matches, and here is every place it could stop"; it is the lexeur that will pick 3.

Remark. The star case is stated as a point fixe in X rather than as a recursive call, and that is not stylistic. The naive reading — "$\mathrm{match}(R*, i)$ is $\{i\}$ together with $\mathrm{match}(R*, j)$ for every $j \in \mathrm{match}(R, i)$" — fails to terminate whenever $\epsilon \in L(R)$, since then $i \in \mathrm{match}(R, i)$ and the recursion calls itself at the same position for ever. Reading it as the *least* point fixe of a monotone map on subsets of $\{0, \dots, n\}$ fixes this: one starts from $\{i\}$ and saturates, adding end positions until nothing new appears. The set is finite and the map only grows, so the iteration stops after at most $n + 1$ rounds.

Remark. Working directly on the expression tree is correct but not cheap: on a word of length n each position may be visited once per node of the tree, and the concatenation case may fan out over a whole set of intermediate positions. This is precisely the inefficiency that the automate view removes.

6.4 Automates

Definition (Automate fini déterministe (*deterministic finite automaton*))**.** Un **automate** (*automaton*) fini déterministe c'est :

- un ensemble d'**états** (*states*) Q ;
- un ensemble d'**états finaux** (*final states*) $F \subseteq Q$;

- un **état initial** (*initial state*) $i \in Q$;
- une **fonction de transition** (*transition function*) $\delta : Q \times \Sigma \to Q$.

Un mot $w_1 \dots w_n$ est **accepté** (*accepted*) par l'automate si $q_n \in F$ avec $q_0 = i$ et $q_{i+1} = \delta(q_i, w_{i+1})$.

Acceptance is thus a single left-to-right pass with one variable holding the état: read a character, replace the état by $\delta(\text{state}, \text{character})$, and at the end ask whether it is an état final. The cost is exactly one transition lookup per character, whatever the expression behind the automate looked like.

Remark. δ is a *function* on all of $Q \times \Sigma$, so it is total: every état has an outgoing transition for every character of the alphabet, and the run never gets stuck — it simply may end outside F. In a drawing one does not draw the transitions that lead nowhere useful; they all go to an implicit non-final **état puits** (*sink state*) that loops on every character. Determinism is the other half of the definition: one target état per pair, hence one run per word, hence no backtracking.

Example (The two automates behind the example lexeur)**.** The rules `[0-9]+` and `[a-z]([a-z]|[0-9])*` of the worked example below become these two automates. Double circles are états finaux; the état puits is not drawn. Both have two useful états, and both spend one transition per character. Compare with running



$[0-9]+$ $\qquad$ $[a-z]([a-z] \mid [0-9])*$

match over the syntax tree of the same expressions: the automate has already done that work, once, at construction time.

Comparaison

Automates	facile à manipuler pour l'ordinateur (union, intersection, test d'appartenance, etc.) ; difficile à comprendre pour un humain
Expressions régulières	facile à comprendre pour un humain ; moins efficace algorithmiquement

Conclusion. Ce que l'on veut : écrire des expressions régulières puis les compiler en automates ! One writes the notation that is readable and compiles it into the representation that is fast — which is the same division of labour the whole module is about, one level down.

6.5 Analyse lexicale : le lexeur

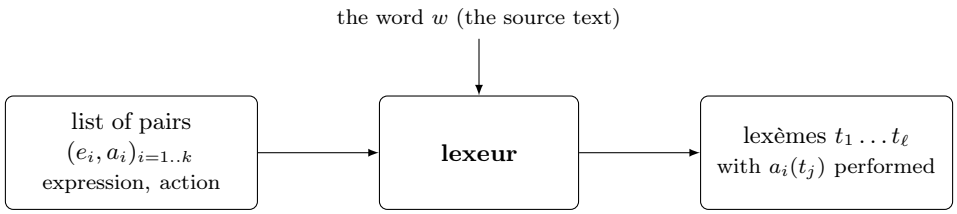
The *lexeur* — the analyseur lexical, the first pass of the compilateur — is the first real consumer of all of this.

Definition (Règles et actions)**.** Une manière de faire des analyseurs lexicaux consiste à utiliser une liste d'expressions régulières $e_1 \dots e_k$ et une liste d'actions $a_1 \dots a_k$. Le lexeur va découper le mot en **lexèmes** $t_1 \dots t_\ell$ et pour le lexème t_j reconnu par e_i il va faire l'action $a_i(t_j)$.

Un exemple

One text is run through one rule table, in order. Texte :

```
let y   = 42 in zzz+1    2+my_var
```

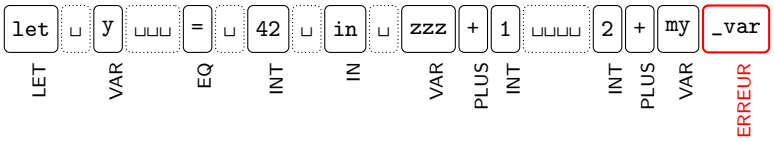


and the rules, *in this order*, are

Expression	Action(x)
$[0-9]+$	Afficher `INT`(x)
`let`	Afficher `LET`
`in`	Afficher `IN`
`=`	Afficher `EQ`
`' '`	Ignorer
`+`	Afficher `PLUS`
$[a-z]([a-z] \mid [0-9])*$	Afficher `VAR`(x)

Affichage :

`LET`, `VAR("y")`, `EQ`, `INT("42")`, `IN`, `VAR("zzz")`, `PLUS`,
`INT("1")`, `INT("2")`, `PLUS`, `VAR("my")`, **ERREUR**.



Dotted boxes are the matches whose action is *Ignorer*; they consume input and emit nothing. The run stops at the underscore: no rule of the table matches a prefix beginning with `_`, so `my_var` comes out as `VAR("my")` followed by an error rather than as one identifier.

L'algorithme, formellement

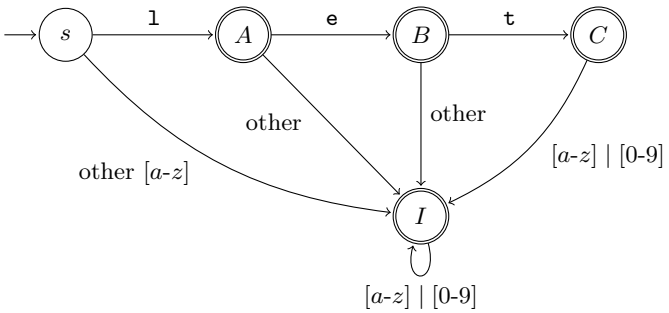
Definition (La boucle du lexeur)**.** Formellement, l'analyseur procède sur le mot $w = w_1 \dots w_n$ de la façon suivante :

- s'il a fini de lire le mot ($n = 0$) il s'arrête ;
- sinon il regarde le **plus grand préfixe** $w_1 \dots w_k$ du mot qui est reconnu par une des expressions e_i ;
- si aucune expression ne reconnaît de préfixe, on déclenche une erreur ;
- parmi les expressions e_i qui reconnaissent le préfixe $w_1 \dots w_k$, il choisit celle avec le i le **plus petit** ;
- il effectue $a_i(w_1 \dots w_k)$;
- et continue sur le mot $w_{k+1} \dots w_n$.

Two tie-breaks, applied in this order and only in this order. First the length: the longest prefix matched by *any* rule wins, whichever rule that is — le **plus grand préfixe** (*maximal munch*). Only then, among the rules that match that same longest prefix, the earliest in the list wins — la **priorité de la règle** (*rule priority*). Everything a lexeur specification does is arranged by these two.

Example (Priorité de la règle : why `let` is not a variable)**.** At the start of the text, the prefix `let` is matched by rule 2 (`let`) and by rule 7 ($[a-z]([a-z] \mid [0-9])*$), both of length 3, and no rule matches anything longer — the next character is a space. Length does not separate them; the priorité de la règle does, and rule 2 wins, so the lexème is `LET` and not `VAR("let")`. A keyword is simply a rule placed above the identifier rule.

Example (Plus grand préfixe : why `lettre` is not a keyword)**.** Reverse the situation and feed the lexeur the text `lettre`. Rule 2 matches the prefix `let`, of length 3; rule 7 matches `lettre`, of length 6. The first tie-break is the length, and $6 > 3$, so the priorité de la règle is never consulted: the lexème is `VAR("lettre")`. This is the whole reason the length test comes first — with the priorité de la règle first, no identifier could ever begin with a keyword.



The picture is the two rules `let` and $[a-z]([a-z] \mid [0-9])*$ merged into one automate, which is what a lexeur generator actually builds; the edges marked "other" carry every letter or digit except the one that continues the keyword. The états A, B, I are final for the identifier rule only, so reaching them means `VAR`; C is final for both rules at once, and it is there that the priorité de la règle decides in favour of `LET`. The scanner does not test the rules one after another — it runs this single automate, remembers the last état final it passed and which rules were satisfied there, and on getting stuck falls back to that position. That is the plus grand préfixe and the priorité de la règle, compiled.

Remark. The rule `' '` of the table is a single quoted space — one character, not one or more. Since its action is *Ignorer*, a run of several spaces simply goes round the loop several times, which is why the four-space run in `zzz+1    2` costs four iterations and produces nothing (and the three-space gap before the `=` costs three). The `ocamllex` version below writes `[' ' '\t' '\n']+` instead, folding the repetition into the expression and also accepting tabs and newlines.

Remark. The error case is the only place the lexeur can fail, and it fails on a *character*, not on a structure: it reports that no rule can start here. Everything about how the lexèmes fit together — that `let` must be followed by a variable, that `+` needs two operands — is beyond the lexeur's reach and is the parser's problem.

ocamllex

Rather than write the loop by hand, one declares the rule table in a `.mll` file and lets a generator emit the automate and the scanner.

```
{ open Lexing }

rule token = parse
  | ['0'-'9']+ as s { `INT (int_of_string s) }
  | "let"     { `LET }
  | "in"      { `IN }
  | '+'       { `PLUS }
  | '='       { `EQ }
  | [' ' '\t' '\n']+ { token lexbuf }
  | ['a'-'z']['a'-'z''0'-'9']* as id { `VAR id }
  | eof       { `EOF }
```

À mettre dans `lexer.mll` puis compiler avec `ocamllex lexer.mll` — *utile pour votre compilateur !*

The correspondence with the table of the worked example is exact: each `|` line is a pair (e_i, a_i), the expression on the left in the concrete syntax of the tool, the action on the right as an OCaml expression between braces. Three points are specific to the tool.

- `as s` binds the matched text — what the definition calls $w_1 \dots w_k$, the argument x of the action — to a variable, so `INT` and `VAR` can carry it. Rules whose action ignores the text, such as `"let"`, need no binding.
- The whitespace rule's action is `token lexbuf`: it calls the scanner again on the rest of the input and returns whatever that returns. A recursive call *is* how one writes *Ignorer* when every rule has to produce a lexème.
- `eof` is an extra pattern the tool provides for end of input, turning the loop's termination condition into an ordinary rule; here it yields `` `EOF ``, a lexème the parser can match on.

Remark. The results are backtick-prefixed — `` `INT ``, `` `LET `` — so they are OCaml *polymorphic variants*: they need no type declared in advance, which is convenient for a type of lexèmes that grows as the compilateur does. The order of the lines is still load-bearing: `"let"` sits above the identifier rule for exactly the reason of the priorité de la règle example above.

Pourquoi un lexeur ?

Conclusion. Tout ce que le lexeur fait peut souvent être fait dans la grammaire mais ça :

- permet de décomposer le problème ;
- permet d'utiliser des expressions régulières ;
- simplifie beaucoup l'écriture et la lecture des grammaires ;
- diminue la complexité des grammaires.

The first clause is the honest one: a grammaire non contextuelle can generate identifiers and integer literals character by character, so the lexeur is not a necessity. It is a separation of concerns that pays. A grammar written over lexèmes rather than characters has no rules for whitespace anywhere, does not have to encode the plus grand préfixe as grammar rules, and — since the langages réguliers are a strict subclass of the languages of grammaires non contextuelles — hands the cheap part of the work to the cheap machine.

6.6 Écrire un lexeur

Example (Objectif)**.** Vous devez écrire du code qui implémente un petit lexeur, c'est-à-dire qui :

- récupère une liste de paires (expression régulière, nom) ;
- une chaîne de caractères ;
- et renvoie le résultat de l'analyse lexicale.

Travailler avec méthode : vous pouvez écrire les fonctions dans l'ordre en testant chacune ; pensez à l'algorithme et s'il est correct ; réfléchissez à ce qui est nécessaire au niveau du code.

L'interface du lexeur

Two extension files, `parser.ml` and `parser.py` — the continuation *pour aller plus loin*, described in the last remark of this chapter — are written against the finished lexeur, so they pin down its interface exactly. The rule list is the table of the worked example reordered and slightly widened — a rule for `*` is added, `INT` moves below `ID`, `ident` also accepts capitals and `space` also accepts a tab — with a name in place of an action:

```
rules = [
    (let_tok, "LET"), (in_tok, "IN"), (ope, "EQ"), (opp, "PLUS"),
    (opf, "FOIS"), (ident, "ID"), (digits, "INT"),
    (space, "SPACE")    # as in the worked example
]
```

and it is consumed by a single function:

```
list_tokens = lexer(rules, input_string)
rev_list_of_tokens = [("EOF","")] + list(reversed(
    [e for e in list_tokens if e[0] != "SPACE"]))
```

```
let token = ref ((lexer rules input_string
                  |> List.filter (fun (x,_) -> x != `SPACE))
                 @ [(`EOF,"")])
```

So `lexer` takes the rule list and the input chaîne de caractères and returns a *list of pairs* (name, matched text), in order of appearance — the OCaml version using polymorphic variants `` `LET ``, `` `ID `` as names where the Python version uses chaînes de caractères. Three consequences for the implementation.

- The action of a rule is not a callback: it is just the name attached to the lexème, and the appelant decides what to do. *Ignorer* is therefore not implemented in the lexeur at all — `SPACE` lexèmes are produced like any other and the appelant filters them out, which is exactly what both snippets do before the analyse syntaxique.
- The matched text must be returned alongside the name, since the parser needs `"42"` to build the integer and the identifier's spelling to look it up in its environment (`Hashtbl.create 17` in OCaml, `dict()` in Python).
- The position of a rule in `rules` is the index i of its priorité de la règle, so the list must be traversed in order: `LET` and `IN` precede `ID`, and an implementation that returns the first rule matching *any* prefix, instead of the first rule matching the *longest* prefix, will lex `lettre` as `LET` followed by `ID("tre")`.

Remark. The natural shape of the code follows the two definitions of this chapter, in order: a `match` over the expression tree in the sense of the recognition equations, returning the set of end positions from a given start; then the loop of the lexeur, which at each position asks every rule for its end positions, keeps the largest k over all rules, breaks ties by the smallest i, emits the pair and restarts at k. The error case is the one where every rule returns an empty set.

Remark. Une extension naturelle consiste à utiliser votre lexeur pour faire de l'analyse syntaxique. Vous pouvez utiliser les fichiers `parser.ml` ou `parser.py` à compléter pour avoir un parseur d'un langage contenant des entiers, des expressions `+`, `*` et des variables introduites par la syntaxe : `let ...= ...in ....` The parser evaluates the expression to a number and fails if it is not parsable or uses an undefined variable. The skeleton they give — a mutable list of lexèmes with `peek_type` and `eat`, a hash table for the environment — is a recursive-descent parser of exactly the kind the grammar chapter builds, which is why the grammar it is aimed at is the one that chapter asks to be made LL(1).